HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



BÁO CÁO BÀI TẬP AGENT 02

ĐỀ TÀI: HỆ ĐA TÁC TỬ VÀ HỌC TĂNG CƯỜNG – TÁC TỬ Q-LEARNING SO VỚI TÁC TỬ THEO LUẬT

Lớp:	D23CTPM01
Nhóm:	Nhóm lớp CT – Nhóm 07
Họ và tên:	Nguyễn Duy Nghĩa
Mã sinh viên:	B23DCCN600
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế
Học kỳ:	Học kỳ 1 năm học 2026 – 2027

Hà Nội, Tháng 10/2026

Mục lục

Mở đầu **1**

1 Thiết kế hệ đa tác tử **3**
1.1 Tác tử, môi trường và mô tả PEAS 3
1.2 Phân chia vai trò và hợp đồng thông điệp 4
1.3 Điều phối tập trung và phân tán 5
1.4 Tích hợp mô hình học và kiểm thử hệ thống 6

2 Học tăng cường theo Naeem và cộng sự (2020) **7**
2.1 Quá trình quyết định Markov 7
2.2 Phương trình Bellman 8
2.3 Các họ thuật toán 9
2.4 Ứng dụng và thách thức 11

3 Cơ chế cộng tác của hệ đa tác tử LLM (Tran và cộng sự, 2025) **13**
3.1 Khung phân tích năm chiều 13
3.2 Kiểu quan hệ: hợp tác, cạnh tranh và cạnh–hợp 14
3.3 Chiến lược và cấu trúc truyền thông 15
3.4 Ứng dụng, thách thức và ánh xạ vào hai hệ của báo cáo 16

4 Mô hình thị trường và hai tác tử **17**
4.1 Mô tả bài toán 17
4.2 Trạng thái và hành động 18
4.3 Chuyển đổi khách hàng, doanh số và lợi nhuận 18
4.4 Động lực môi trường 19
4.5 Tác tử Q-learning 19
4.6 Tác tử theo luật 20
4.7 Phân tích lợi nhuận kỳ vọng một bước 20
4.8 Chính sách tối ưu khi đối thủ là tác tử luật 21

5 Cài đặt và thiết kế thí nghiệm **23**
5.1 Kiến trúc chương trình 23
5.2 Các thay đổi so với mã tham chiếu 24
5.3 Tham số tham chiếu 24

5.4 Giao thức đánh giá . . . 25
5.5 Danh mục thí nghiệm và chỉ số đo . . . 26
5.6 Trích đoạn mã nguồn chính . . . 27
5.7 Môi trường chạy và phân chia CPU/GPU . . . 28

6 Kết quả thực nghiệm 29
6.1 Thí nghiệm A: cấu hình tham chiếu . . . 29
6.2 Thí nghiệm B: quá trình học . . . 30
6.3 Thí nghiệm C: tốc độ học . . . 32
6.4 Thí nghiệm D: mức khám phá . . . 33
6.5 Thí nghiệm E: so sánh công bằng trên nhiều seed . . . 34
6.6 Thí nghiệm F (mở rộng): mức giảm giá . . . 36
6.7 Thí nghiệm G (mở rộng): nhu cầu bền và thưởng theo doanh thu 37
6.8 Thí nghiệm H (mở rộng): hai tác tử Q-learning cùng học . . . 37
6.9 Tổng hợp . . . 38

7 Trả lời mười câu hỏi của bài thí nghiệm 39
7.1 Câu 1: Mô hình hóa tác tử và môi trường . . . 39
7.2 Câu 2: Quyết định dựa trên học và dựa trên luật . . . 39
7.3 Câu 3: Tính một bước cập nhật Q-learning . . . 40
7.4 Câu 4: Khám phá và khai thác . . . 40
7.5 Câu 5: Phần thưởng và mục tiêu kinh doanh . . . 41
7.6 Câu 6: Đánh giá hiệu năng từ dữ liệu thực nghiệm . . . 41
7.7 Câu 7: Tốc độ học và tính ổn định . . . 41
7.8 Câu 8: Tương tác đa tác tử và tính phi dừng . . . 42
7.9 Câu 9: Giới hạn và thay đổi môi trường đã thực hiện . . . 42
7.10 Câu 10: Đề xuất và kiểm chứng tác tử tốt hơn . . . 42

8 Hệ đa tác tử hỗ trợ mua sắm 46
8.1 Đặc tả PEAS và luồng thông điệp . . . 46
8.2 Bộ phân loại ý định bằng PyTorch trên GPU . . . 48
8.3 Demo, kiểm thử và mô phỏng lỗi . . . 49

9 Thảo luận, hạn chế và kết luận 51
9.1 Những quan sát chính . . . 51
9.2 Hạn chế . . . 51
9.3 Kết luận . . . 52

Tài liệu tham khảo **53**

Mở đầu

Một cửa hàng thương mại điện tử phải quyết định giá mỗi ngày trong khi nhu cầu không chắc chắn và đối thủ cũng đang quyết định giá. Có hai cách xây dựng "bộ não" cho quyết định đó. Cách thứ nhất là viết luật: người thiết kế đoán trước các tình huống và ghi sẵn phản ứng. Cách thứ hai là cho tác tử tự học từ phần thưởng bằng học tăng cường (Reinforcement Learning – RL). Luật dễ hiểu và an toàn nhưng cứng; RL thích nghi được nhưng tốn dữ liệu, phụ thuộc seed và có thể học sai nếu phần thưởng hoặc trạng thái thiết kế không khớp mục tiêu.

Bài tập Agent 02 đặt hai cách này cạnh nhau trong một hệ đa tác tử (Multi-Agent System – MAS) tối giản: hai cửa hàng, mỗi bước chọn giảm giá hoặc giữ giá [1]. Phần lý thuyết dựa trên ba tài liệu được giao: tutorial phát triển hệ đa tác tử bằng Python và PyTorch [2], bài tổng quan học tăng cường của Naeem và cộng sự [3], và khảo sát cơ chế cộng tác của hệ đa tác tử dựa trên mô hình ngôn ngữ lớn của Tran và cộng sự [4].

Mục tiêu

1. Trình bày cơ sở thiết kế hệ đa tác tử, học tăng cường và cơ chế cộng tác đa tác tử theo ba tài liệu.
2. Mô hình hóa bài toán theo tác tử, môi trường, trạng thái, hành động, phần thưởng, chính sách; tính sẵn chính sách tối ưu làm mốc.
3. Cài đặt môi trường, tác tử Q-learning và tác tử luật bằng Python; tách huấn luyện và đánh giá.
4. Thực hiện năm thí nghiệm A–E của đề bài và ba thí nghiệm mở rộng F–H; trả lời mười câu hỏi bằng số liệu đo được, cài đặt một thay đổi môi trường (câu 9) và một tác tử tốt hơn (câu 10).
5. Hiện thực hệ đa tác tử hỗ trợ mua sắm theo tutorial, với bộ phân loại ý định PyTorch huấn luyện trên GPU.

Cách đọc báo cáo

Chương 1–3 là lý thuyết. Chương 4 hình thức hóa bài toán, Chương 5 mô tả chương trình và giao thức đánh giá, Chương 6 trình bày kết quả, Chương 7 trả lời mười câu hỏi, Chương 8 là hệ mua sắm đa tác tử, Chương 9 thảo luận và kết luận. Mọi con số thực nghiệm trong báo cáo được sinh tự động từ tệp JSON do chương trình ghi; mọi đoạn mã được trích nguyên văn từ mã đã chạy.

Kết luận chính

Ở cấu hình của đề bài ($\delta = 20\%$), chính sách tối ưu khi đối thủ là tác tử luật là *không bao giờ giảm giá*, đạt 35 mỗi bước, và tác tử luật vốn đã làm đúng như vậy. Q-learning học được chính sách tối ưu ở $9/10$ seed; chênh lệch so cặp với luật là $-0{,}7553$ với khoảng tin cậy 95% $[-2{,}464;\ 0{,}9532]$, chứa 0. Khi giảm giá rẻ hơn ($\delta = 5\%$) luật không còn tối ưu và RL vượt luật $1{,}85$ mỗi bước. Tác tử dựa trên mô hình đề xuất ở câu 10 đạt tối ưu ở cả năm kịch bản thử.

Chương 1

Thiết kế hệ đa tác tử

Chương này tóm lược tài liệu hướng dẫn của học phần *Developing Multiagent Systems: A Practical Tutorial with Python and PyTorch* [2], bổ sung bằng hai giáo trình kinh điển về tác tử [5, 6]. Mục tiêu là xác lập bộ khái niệm (tác tử, môi trường, PEAS, vai trò, thông điệp, điều phối, kiểm thử) mà Chương 4 dùng để mô hình hóa thị trường hai cửa hàng và Chương 8 dùng để cài đặt hệ mua sắm.

1.1 Tác tử, môi trường và mô tả PEAS

Định nghĩa 1.1 (Tác tử và hệ đa tác tử)

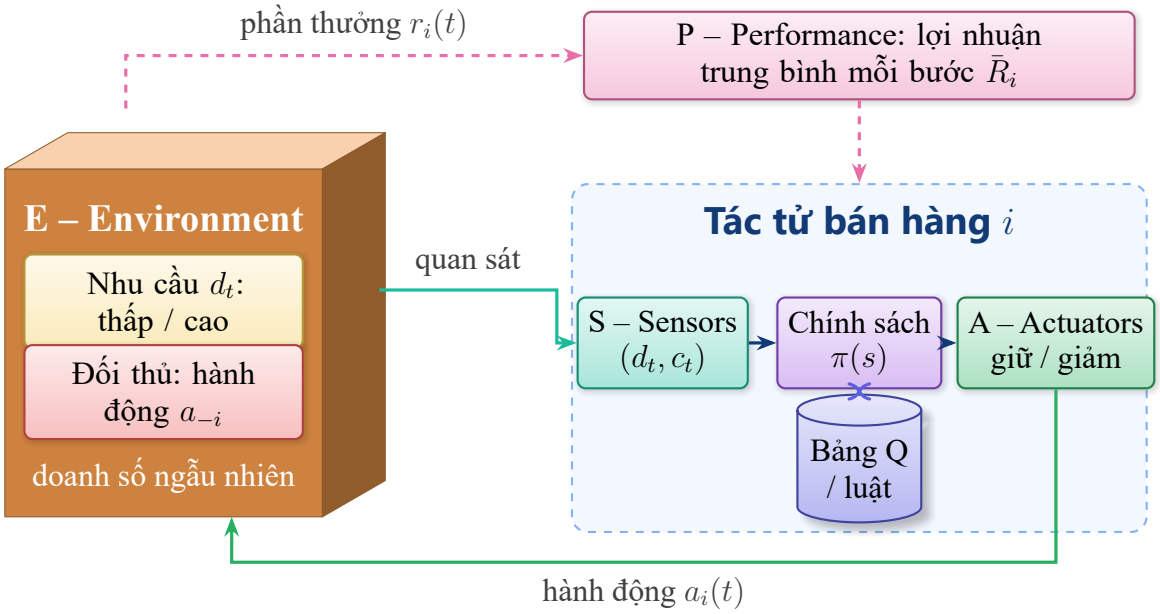
Tác tử (agent) là một thành phần tính toán nhận thông tin từ môi trường qua cảm biến (sensors), chọn hành động theo mục tiêu và tác động trở lại môi trường qua cơ cấu chấp hành (actuators). **Hệ đa tác tử** (multi-agent system, MAS) gồm từ hai tác tử trở lên có quyết định ảnh hưởng lẫn nhau, qua thông điệp, qua trạng thái dùng chung hoặc qua chính môi trường [5].

Điểm quan trọng của định nghĩa là các tác tử *không nhất thiết trò chuyện* với nhau. Hai cửa hàng trực tuyến không gửi thông điệp nào cho nhau, nhưng quyết định giảm giá của cửa hàng này làm thay đổi xác suất mua hàng ở cửa hàng kia; tương tác gián tiếp qua môi trường như vậy vẫn tạo thành một MAS.

Để đặc tả một tác tử trước khi lập trình, tutorial dùng khung **PEAS** [6]: *Performance* (tiêu chí đánh giá), *Environment* (môi trường hoạt động), *Actuators* (cách tác động) và *Sensors* (thông tin nhận được). Hình 1.1 đặt PEAS lên vòng cảm nhận–hành động của tác tử bán hàng trong bài thí nghiệm.

Nhận xét: Cùng một vòng lặp áp dụng cho cả hai tác tử của bài thí nghiệm; chúng chỉ khác ở khối "chính sách": tác tử học tăng cường cập nhật chính sách từ phần thưởng, tác tử theo luật giữ nguyên một bảng if–else.

Ngoài PEAS, tutorial nêu năm **thuộc tính** thường gặp của tác tử: tự chủ (autonomy), phản ứng (reactivity), chủ động (pro-activeness), giao tiếp (social



Hình 1.1. Vòng cảm nhận – quyết định – hành động của tác tử bán hàng, đánh dấu bốn thành phần PEAS.

Bảng 1.1. Đặc tả PEAS của tác tử bán hàng trong bài thí nghiệm giảm giá.

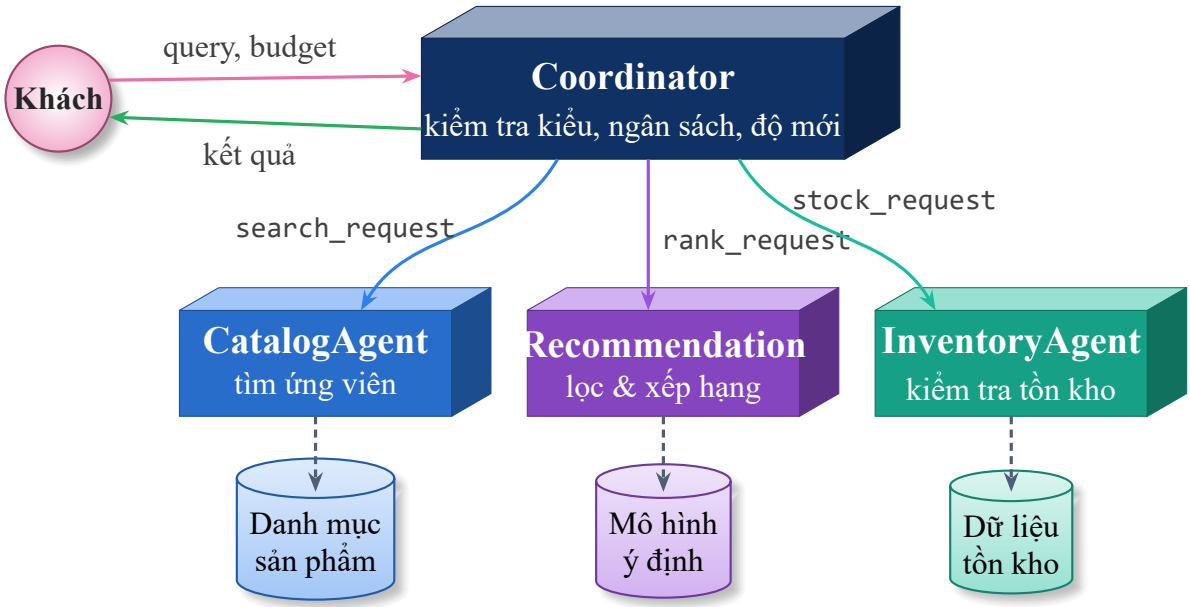
Thành phần	Nội dung
Performance	Lợi nhuận trung bình mỗi bước trong pha đánh giá; phụ: tỷ lệ giảm giá, độ ổn định qua các seed
Environment	Thị trường mô phỏng: nhu cầu ngẫu nhiên hai mức, một đối thủ, số khách và doanh số ngẫu nhiên
Actuators	Hai hành động: giữ giá hoặc giảm giá trong bước hiện tại
Sensors	Mức nhu cầu hiện tại và hành động của đối thủ ở bước trước

ability) và học (learning). Các thuộc tính này xuất hiện ở mức độ khác nhau: tác tử theo luật vẫn tự chủ và phản ứng với trạng thái mới nhưng không học; tác tử Q-learning học trong pha huấn luyện rồi chỉ thực thi chính sách đã học trong pha đánh giá. Sự tách hai pha này là nguyên tắc xuyên suốt của Chương 5.

1.2 Phân chia vai trò và hợp đồng thông điệp

Một MAS được thiết kế tốt bắt đầu bằng việc chia trách nhiệm thành các **vai trò** (roles) hẹp, mỗi vai trò có đầu vào và đầu ra xác định. Hệ mua sắm của tutorial gồm bốn vai trò [2]: **CatalogAgent** tìm sản phẩm ứng viên theo truy vấn; **RecommendationAgent** lọc theo ngân sách và xếp hạng; **InventoryAgent** xác

minh tồn kho; **Coordinator** nhận yêu cầu, gọi lần lượt các vai trò và kiểm tra kết quả trước khi trả lời người dùng (Hình 1.2).



Hình 1.2. Hệ mua sắm bốn vai trò của tutorial: mọi lời gọi đi qua Coordinator; mỗi mũi tên là một thông điệp có kiểu.

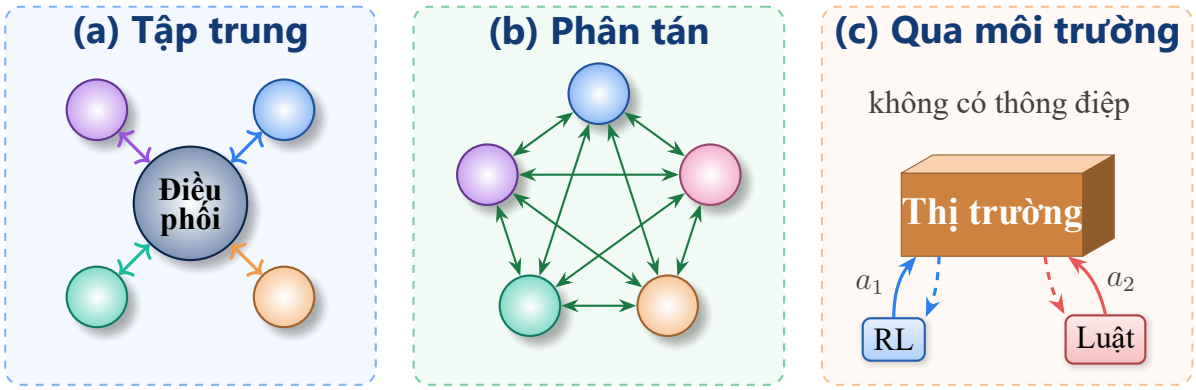
Nhận xét: Coordinator là điểm kiểm soát duy nhất: nó kiểm tra người nhận, kiểu phản hồi, ngân sách và độ mới của dữ liệu tồn kho, nên một tác tử trả lời sai bị chặn trước khi lan sang bước sau.

Các tác tử trao đổi qua **thông điệp** có cấu trúc cố định. Tutorial dùng bốn trường bắt buộc: `sender`, `receiver`, `kind` (loại thông điệp, ví dụ `search_request`, `rank_response`) và `payload` (dữ liệu). Hệ lớn hơn bổ sung mã tác vụ, mã thông điệp, thời điểm gửi và hạn chót để truy vết. Cố định hợp đồng thông điệp có hai lợi ích: mỗi tác tử kiểm thử được độc lập bằng thông điệp giả, và Coordinator phát hiện được phản hồi sai kiểu thay vì lan truyền lỗi. Chương 8 viết hợp đồng này thành kiểu dữ liệu (`dataclass`) được kiểm tra khi nhận.

1.3 Điều phối tập trung và phân tán

Cơ chế **điều phối** (coordination) quyết định ai gọi ai, theo thứ tự nào và ai giải quyết xung đột. Điều phối tập trung dễ theo dõi, dễ kiểm thử, nhưng nút điều phối có thể thành điểm nghẽn và điểm lỗi đơn. Điều phối phân tán tăng quyền tự

chủ và khả năng mở rộng, đổi lại phải xử lý xung đột, tính nhất quán và lỗi giao tiếp (Hình 1.3). Hai cửa hàng trong bài thí nghiệm thuộc dạng thứ ba: không có điều phối viên, mỗi tác tử quyết định độc lập và đồng thời, môi trường áp dụng cả hai hành động rồi trả phần thưởng riêng.



Hình 1.3. Ba dạng điều phối: tập trung (hệ mua sắm), phân tán ngang hàng, và tương tác gián tiếp qua môi trường (thị trường hai cửa hàng).

Nhận xét: Dạng điều phối quyết định bài toán học: ở dạng thứ ba, mỗi tác tử thấy đối thủ như một phần của môi trường; nếu đối thủ cũng học, môi trường trở nên phi dừng (Chương 2, Mục 2.4).

1.4 Tích hợp mô hình học và kiểm thử hệ thống

Mạng nơ-ron trong MAS chỉ là một thành phần dự đoán nhãn (ví dụ ý định của khách) hoặc điểm xếp hạng; tutorial minh họa bằng mạng `IntentNet` (PyTorch) và yêu cầu đánh giá nó riêng trên tập kiểm tra [2].

Kiểm thử MAS vì vậy có ba tầng: (i) *kiểm thử đơn vị* từng tác tử với thông điệp giả; (ii) *kiểm thử tích hợp* luồng qua Coordinator, gồm cả *mô phỏng lỗi* (tác tử im lặng, trả sai kiểu, dữ liệu tồn kho cũ); (iii) *đánh giá hệ thống* bằng chỉ số theo mục tiêu: tỷ lệ hoàn thành tác vụ, số lần vi phạm ngân sách, số xác nhận tồn kho chưa xác minh, độ trễ và số thông điệp. Với bài thí nghiệm giảm giá, chỉ số hệ thống là lợi nhuận đo trên pha đánh giá tách biệt, nhiều seed; Bảng 3.1 ở Chương 3 so sánh đầy đủ hai hệ.

Chương 2

Học tăng cường theo Naeem và cộng sự (2020)

Bài tổng quan *A Gentle Introduction to Reinforcement Learning and its Application in Different Fields* của Naeem, Rizvi và Coronato (IEEE Access, tập 8, tr. 209320–209344) [3] hệ thống hóa học tăng cường (reinforcement learning, RL) theo ba lớp: khung toán học MDP (phần II–III), các họ thuật toán (phần IV) và ứng dụng cùng thách thức (phần V). Chương này đi theo đúng trình tự đó, đối chiếu với giáo trình của Sutton và Barto [7], và chỉ ra phần nào được dùng trực tiếp trong thí nghiệm ở Chương 4–7.

2.1 Quá trình quyết định Markov

Định nghĩa 2.1 (MDP)

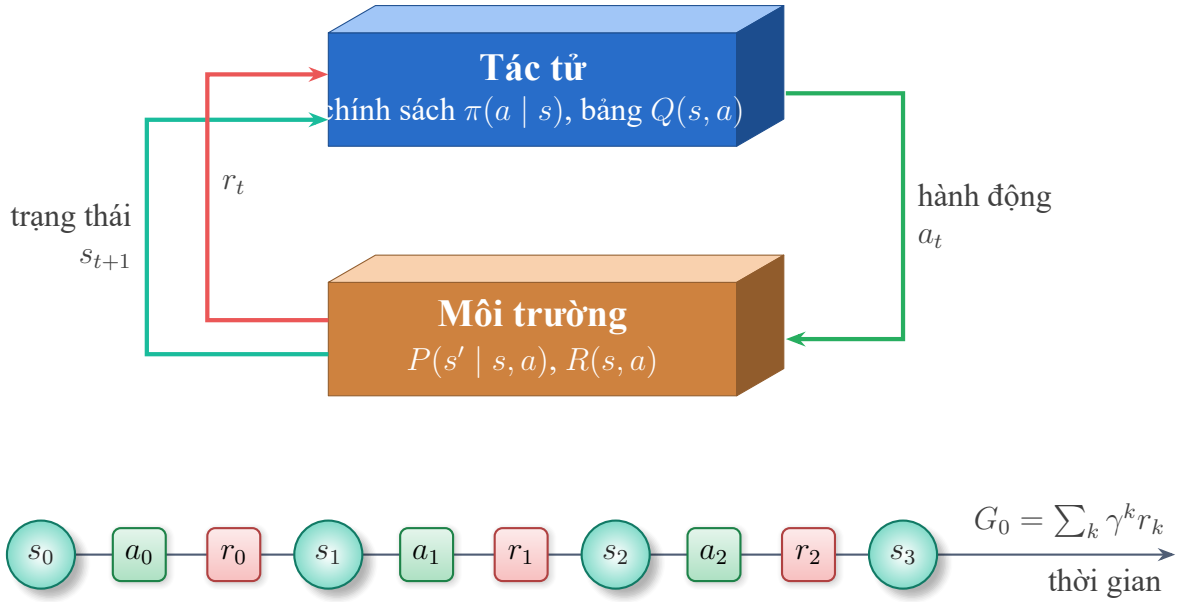
Một quá trình quyết định Markov (Markov decision process) là bộ $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ gồm tập trạng thái $\mathcal{S}$, tập hành động $\mathcal{A}$, xác suất chuyển $P(s' \mid s, a)$, hàm phần thưởng $R(s, a)$ và hệ số chiết khấu $\gamma \in [0, 1)$. Tính Markov yêu cầu $\Pr(s_{t+1}, r_t \mid s_t, a_t, s_{t-1}, \dots) = \Pr(s_{t+1}, r_t \mid s_t, a_t)$.

Tác tử và môi trường tương tác theo các bước rời rạc (Hình 2.1): ở bước t tác tử thấy s_t, chọn $a_t \sim \pi(\cdot \mid s_t)$, nhận phần thưởng r_t và trạng thái mới s_{t+1}. Mục tiêu không phải phần thưởng tức thời mà là **lợi tức** (return) chiết khấu, và chất lượng của chính sách được đo bằng hàm giá trị:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}, \tag{2.1}$$

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s], \qquad Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a].$$

Nhận xét: Phần thưởng r_t là phản hồi một bước, còn G_t cộng dồn có chiết khấu.



Hình 2.1. Vòng tương tác tác tử – môi trường của MDP và quỹ đạo $(s_0, a_0, r_0, s_1, \dots)$ sinh ra theo thời gian.

Trong bài thí nghiệm, báo cáo lợi nhuận *trung bình không chiết khấu* mỗi bước, trong khi bảng Q ước lượng lợi tức với $\gamma = 0{,}9$; hai đại lượng vì vậy khác thang đo dù cùng đơn vị tiền.

2.2 Phương trình Bellman

Tách số hạng đầu khỏi tổng (2.1) cho quan hệ đệ quy giữa giá trị hiện tại và giá trị ở bước sau. Với một chính sách cố định π ta có **phương trình Bellman kỳ vọng**, còn với chính sách tối ưu ta có **phương trình Bellman tối ưu** [3, 7]:

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s'} P(s' \mid s, a)\big[R(s,a) + \gamma V^{\pi}(s')\big], \tag{2.2}$$

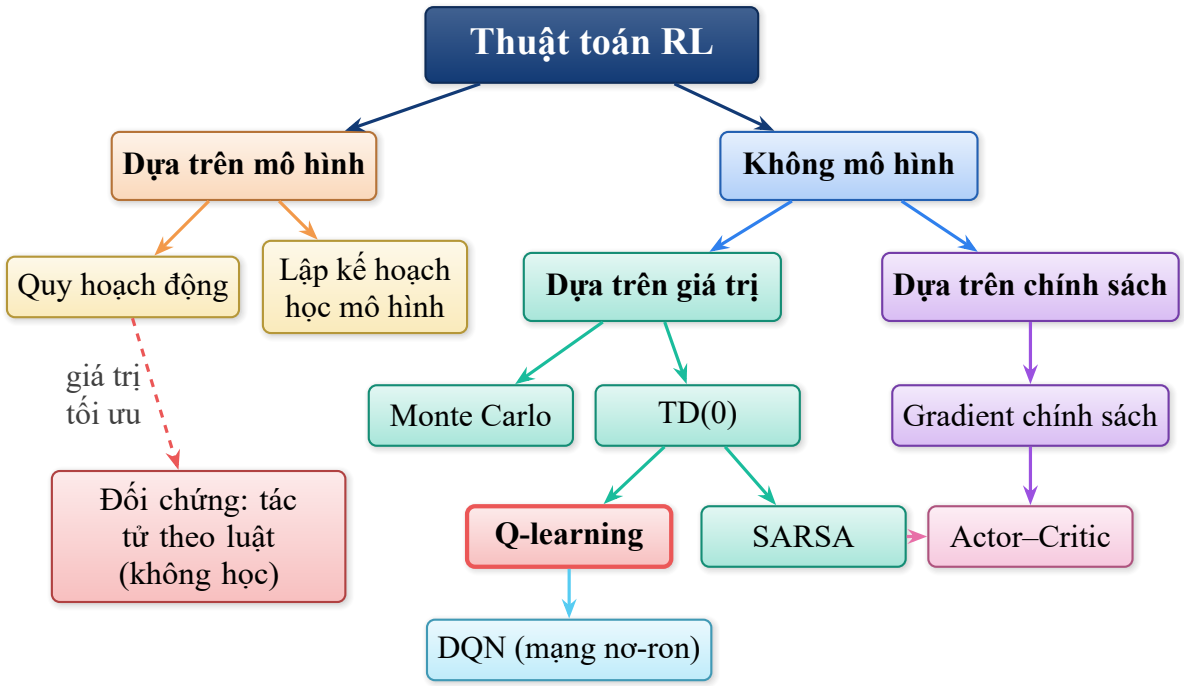
$$Q^{*}(s,a) = \sum_{s'} P(s' \mid s, a)\Big[R(s,a) + \gamma \max_{a'} Q^{*}(s', a')\Big], \tag{2.3}$$

$$\pi^{*}(s) = \arg\max_{a} Q^{*}(s,a).$$

Khi biết P và R, (2.3) giải được bằng lặp giá trị; khi không biết, các thuật toán RL thay kỳ vọng bằng mẫu trải nghiệm. Bài thí nghiệm nhỏ đến mức mô hình P, R tính được chính xác, nên Chương 4 dùng (2.3) làm giá trị đối chiếu cho những gì Q-learning học được.

2.3 Các họ thuật toán

Naeem và cộng sự chia thuật toán theo hai trục: có dùng mô hình môi trường hay không (model-based / model-free), và học giá trị hay học trực tiếp chính sách (Hình 2.2).



Hình 2.2. Phân loại các thuật toán RL theo bài tổng quan; ô viền đỏ là thuật toán được dùng trong thí nghiệm.

- **Quy hoạch động** (dynamic programming, DP): lặp chính sách / lặp giá trị trên mô hình đã biết; chính xác nhưng cần P và tốn kém khi $|\mathcal{S}|$ lớn.
- **Monte Carlo** (MC): ước lượng V^{π} bằng trung bình lợi tức của các episode hoàn chỉnh; không *bootstrap*, không thiên lệch nhưng phương sai lớn và phải chờ hết episode.
- **Sai phân thời gian** (temporal difference, TD): cập nhật sau từng bước bằng ước lượng của chính nó, $V(s_t) \leftarrow V(s_t) + \alpha\,[r_t + \gamma V(s_{t+1}) - V(s_t)]$; kết hợp lấy mẫu của MC với bootstrap của DP.
- **Actor–Critic**, phương pháp Bayes và **DQN**: tách chính sách (actor) khỏi giá trị (critic), mô hình hóa bất định, hoặc xấp xỉ Q bằng mạng nơ-ron cho không gian lớn.

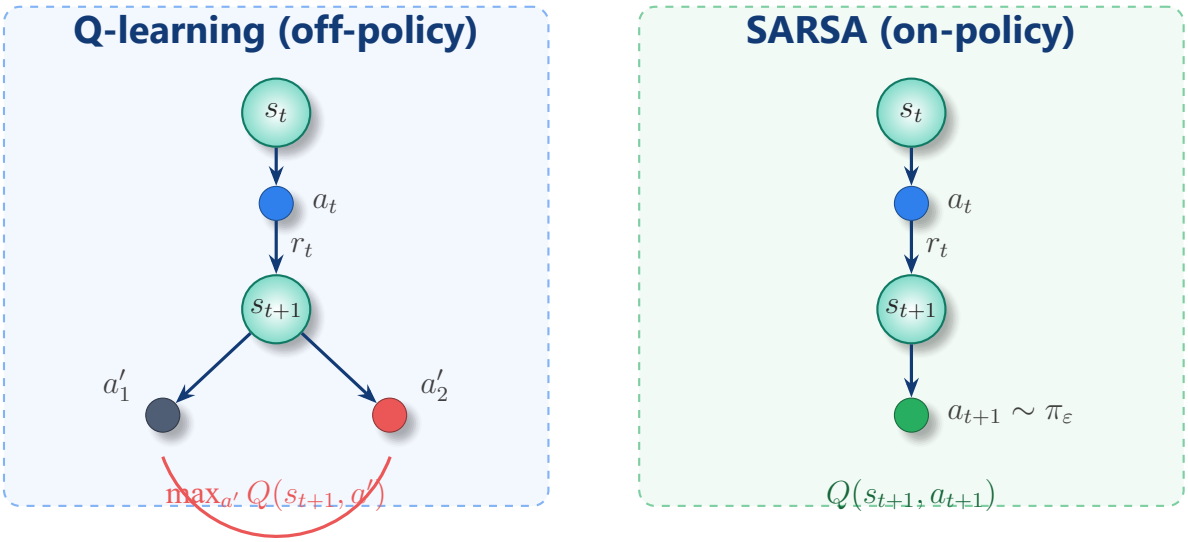
2.3.1 Q-learning và SARSA

Hai đại diện quan trọng nhất của TD điều khiển chỉ khác nhau ở *mục tiêu* (target) của phép cập nhật (Hình 2.3). Q-learning [8] là **off-policy**: mục tiêu lấy hành động tốt nhất ở trạng thái sau, bất kể hành động thực sự được chọn; SARSA là **on-policy**: mục tiêu dùng chính hành động a_{t+1} mà chính sách đang khám phá chọn:

$$\text{Q-learning:}\quad Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha\Big[\underbrace{r_t + \gamma \max_{a'} Q(s_{t+1},a')}_{y_t} - Q(s_t,a_t)\Big], \tag{2.4}$$

$$\text{SARSA:}\quad Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha\big[r_t + \gamma\, Q(s_{t+1},a_{t+1}) - Q(s_t,a_t)\big]. \tag{2.5}$$

Ở bước cuối episode, mục tiêu chỉ còn $y_t = r_t$. Đại lượng $\delta_t = y_t - Q(s_t,a_t)$ là **sai số TD**.



Hình 2.3. Sơ đồ sao lưu (backup diagram) của Q-learning (lấy max) và SARSA (lấy hành động thực).

Nhận xét: Vì Q-learning học giá trị của chính sách tham lam trong khi vẫn khám phá, nó cho phép tách gọn pha huấn luyện ($\varepsilon > 0$) khỏi pha đánh giá ($\varepsilon = 0$) — đúng cách đánh giá được dùng ở Chương 5.

2.3.2 Khám phá, tốc độ học và điều kiện hội tụ

Chiến lược ε-**greedy** chọn ngẫu nhiên với xác suất ε, ngược lại chọn $\arg\max_a Q(s,a)$; với hai hành động, hành động "không tốt nhất" được chọn với xác suất $\varepsilon/2$. Thường ε giảm dần, ví dụ $\varepsilon_e = \max(\varepsilon_{\min}, \varepsilon_0 \rho^e)$ theo episode e. Tốc độ học α quyết định mỗi mẫu thay đổi Q bao nhiêu: α nhỏ cho đường học mượt nhưng chậm, α lớn phản ứng nhanh nhưng khuếch đại nhiễu. Q-learning hội tụ về Q^* với xác suất 1 nếu MDP dừng, mọi cặp (s,a) được thăm vô hạn lần và $\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$ [8].

Khoảng cách giữa lý thuyết và thí nghiệm

Thí nghiệm dùng α hằng, số bước hữu hạn và một trạng thái rút gọn không chứa hành động trước của chính tác tử; đối thủ cũng có thể học. Cả ba điều kiện trên vì vậy không được bảo đảm hoàn toàn, nên độ ổn định phải được đo bằng nhiều seed thay vì suy ra từ định lý.

2.3.3 Từ bảng Q tới DQN

Khi $|\mathcal{S}|$ lớn, bảng Q được thay bằng mạng $Q(s,a;\theta)$. DQN [9] ổn định quá trình học bằng hai cơ chế: *bộ nhớ phát lại* (experience replay) phá tương quan giữa các mẫu liên tiếp, và *mạng mục tiêu* θ^- cập nhật chậm; hàm mất mát là $\mathcal{L}(\theta) = \mathbb{E}\big[(r + \gamma \max_{a'} Q(s',a';\theta^-) - Q(s,a;\theta))^2\big]$. Bài thí nghiệm chỉ có $4 \times 2 = 8$ giá trị Q nên bảng là lựa chọn đúng; DQN được nêu để chỉ hướng mở rộng khi trạng thái giàu hơn.

2.4 Ứng dụng và thách thức

Phần V của bài tổng quan khảo sát RL trong nhiều lĩnh vực; điểm chung là chuỗi quyết định nối tiếp, trong đó hành động hôm nay thay đổi tình huống ngày mai (Bảng 2.1).

Các thách thức được nêu gồm: **hiệu quả mẫu** (cần rất nhiều tương tác), **không gian lớn**, **an toàn** khi khám phá trên hệ thật, **quan sát một phần**, **môi trường phi dừng** và đánh đổi **thiên lệch–phương sai**. Cả sáu đều xuất hiện ở quy mô nhỏ trong bài thí nghiệm: doanh số nhị thức tạo phương sai; bootstrap

Bảng 2.1. Một số lĩnh vực ứng dụng RL được Naeem và cộng sự khảo sát.

Lĩnh vực	Trạng thái / hành động	Phần thưởng điển hình
Robot, điều khiển	Cảm biến khớp / mô-men xoắn	Bám quỹ đạo, tiết kiệm năng lượng
Mạng, viễn thông	Tải kênh / cấp phát tài nguyên	Thông lượng, độ trễ
IoT tự chủ	Trạng thái thiết bị / lịch hoạt động	Tuổi thọ pin, chất lượng dịch vụ
Ngôn ngữ, thị giác	Ngữ cảnh / từ hoặc vùng chọn	Điểm chất lượng đầu ra
Trò chơi	Bàn cờ, khung hình / nước đi	Thắng / thua, điểm
Lập lịch, tài nguyên	Hàng đợi / phân công	Thời gian hoàn thành
Y tế	Tình trạng bệnh nhân / phác đồ	Kết cục điều trị

tạo thiên lệch; trạng thái thiếu hành động trước của chính mình làm quan sát không đầy đủ; nếu đối thủ cũng học, phân phối phản hồi thay đổi theo thời gian; và mỗi lần "khám phá" bằng giảm giá là một khoản lợi nhuận thật bị mất. Vì vậy thí nghiệm lấy từ bài tổng quan bốn thành phần: MDP (mô hình hóa), Q-learning off-policy với ε-greedy giảm dần (tác tử A), Bellman tối ưu trên mô hình đã biết (giá trị đối chiếu), và thiết kế đánh giá nhiều seed cùng thí nghiệm hai tác tử cùng học (đối phó phương sai và tính phi dừng).

Chương 3

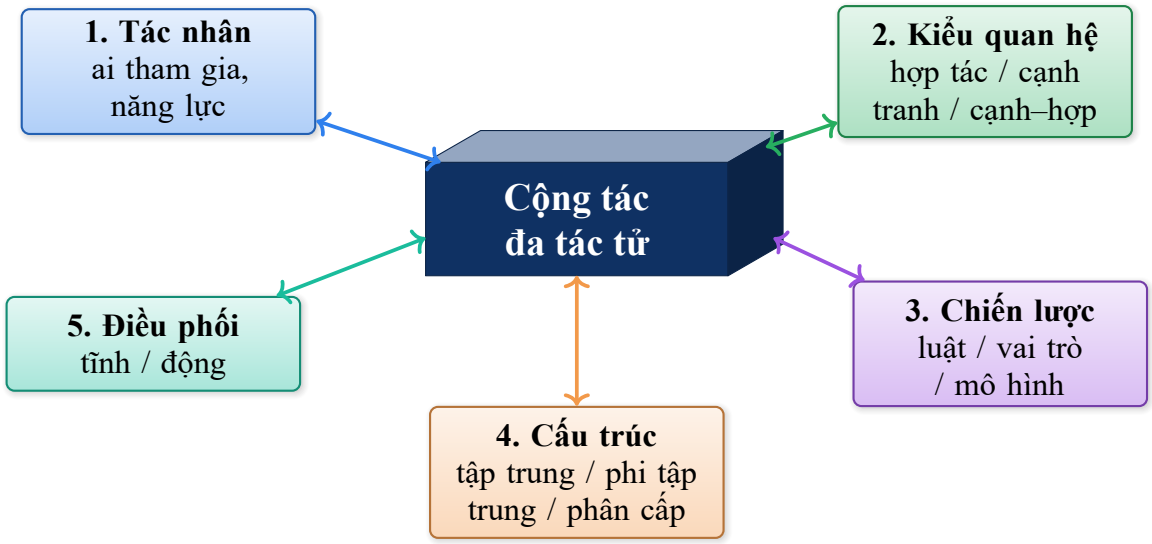
Cơ chế cộng tác của hệ đa tác tử LLM (Tran và cộng sự, 2025)

Bài khảo sát *Multi-Agent Collaboration Mechanisms: A Survey of LLMs* của Tran, Dao, Nguyen, Pham, O'Sullivan và Nguyen (arXiv:2501.06322, 2025) [4] chuyển trọng tâm từ năng lực của một mô hình ngôn ngữ lớn (LLM) sang câu hỏi: *nhiều tác tử phối hợp với nhau như thế nào*. Dù bài thí nghiệm không dùng LLM, khung phân tích của bài khảo sát đủ tổng quát để mô tả cả thị trường hai cửa hàng lẫn hệ mua sắm của Chương 8.

3.1 Khung phân tích năm chiều

Bài khảo sát mô tả một hệ cộng tác bằng năm chiều (Hình 3.1): **tác nhân** (actors) — ai tham gia; **kiểu quan hệ** (types) — hợp tác, cạnh tranh hay vừa hợp tác vừa cạnh tranh; **chiến lược** (strategies) — dựa trên luật, vai trò hay mô hình; **cấu trúc truyền thông** (structures) — tập trung, phi tập trung hay phân cấp; và **điều phối** (coordination/orchestration) — tĩnh hay động. Mỗi chiều vừa là nguồn lợi ích vừa là nguồn chi phí: thêm một tác tử kiểm chứng giảm sai sót nhưng làm tăng số lời gọi và độ trễ.

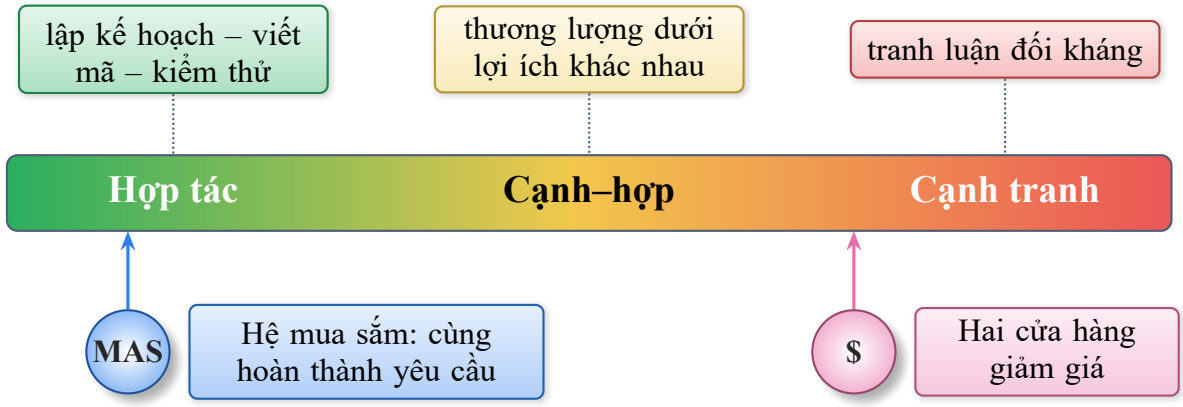
Nhận xét: Năm chiều độc lập với nhau: cùng một tập tác tử có thể ghép với nhiều cấu trúc và cách điều phối khác nhau, nên thiết kế MAS là chọn một điểm trong không gian năm chiều này chứ không chọn một "kiến trúc mẫu".



Hình 3.1. Khung năm chiều mô tả cơ chế cộng tác đa tác tử theo Tran và cộng sự.

3.2 Kiểu quan hệ: hợp tác, cạnh tranh và cạnh–hợp

Hợp tác (cooperation) là cùng đóng góp vào một mục tiêu chung, ví dụ nhóm tác tử lập kế hoạch, viết mã và kiểm thử. **Cạnh tranh** (competition) là ưu tiên mục tiêu riêng có thể xung đột, ví dụ tranh luận đối kháng để lộ điểm yếu của lập luận. **Cạnh–hợp** (coopetition) kết hợp hai dạng trên, như thương lượng tìm thỏa thuận dưới lợi ích khác nhau [4]. Hình 3.2 đặt hai hệ của báo cáo lên trục này.



Hình 3.2. Phổ kiểu quan hệ giữa các tác tử và vị trí của hai hệ trong báo cáo.

Nhận xét: Thị trường hai cửa hàng là cạnh tranh nhưng *không* tổng bằng không: tổng lợi nhuận thay đổi theo số đơn và biên lãi, nên cả hai cùng giữ giá có thể tốt hơn cho cả hai so với cùng giảm giá — đúng dạng thế lưỡng nan mà Chương 4

sẽ tính bằng số.

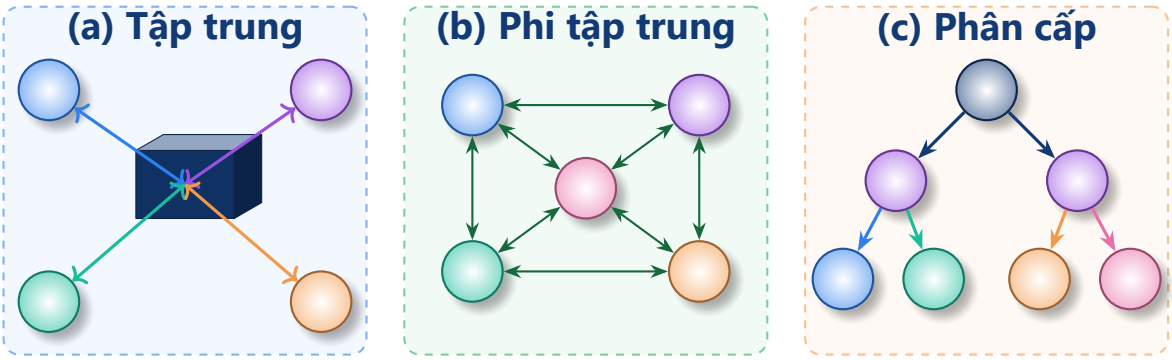
3.3 Chiến lược và cấu trúc truyền thông

Ở chiều chiến lược, bài khảo sát phân biệt ba loại giao thức. **Dựa trên luật** (rule-based): hành vi định sẵn bằng quy tắc, dễ dự đoán và kiểm tra nhưng kém thích nghi. **Dựa trên vai trò** (role-based): mỗi tác tử giữ một vai chuyên biệt, hỗ trợ chuyên môn hóa và phân công. **Dựa trên mô hình** (model-based): tác tử suy luận xác suất về môi trường và ý định của tác tử khác, xử lý được bất định nhưng đắt và khó triển khai.

Hai nghĩa của "model-based"

"Model-based" trong phân loại cộng tác chỉ việc suy luận về tác tử khác; nó không đồng nghĩa với model-based RL (học mô hình chuyển trạng thái). Q-learning của bài thí nghiệm là model-free, còn tác tử theo luật là rule-based.

Ở chiều cấu trúc, các tác tử có thể giao tiếp qua một nút trung tâm (**tập trung**), trực tiếp với nhau (**phi tập trung**, ngang hàng) hoặc theo nhiều tầng quản lý (**phân cấp**) (Hình 3.3). Cấu trúc tập trung dễ kiểm soát nhưng có điểm nghẽn; phi tập trung bền với lỗi một nút nhưng tốn thông điệp và khó đạt đồng thuận; phân cấp cân bằng hai bên cho bài toán lớn. Về điều phối, luồng **tĩnh** định trước thứ tự các bước, còn luồng **động** chọn tác tử và phụ thuộc khi đã biết yêu cầu cụ thể.



Hình 3.3. Ba cấu trúc truyền thông: tập trung, phi tập trung và phân cấp.

3.4 Ứng dụng, thách thức và ánh xạ vào hai hệ của báo cáo

Bài khảo sát tổng hợp ứng dụng trong mạng 5G/6G và Công nghiệp 5.0, hỏi đáp và sinh ngôn ngữ, cùng các mô phỏng xã hội và văn hóa. Kết luận chung là hiệu quả phụ thuộc vào kênh phối hợp, tri thức miền và chi phí mở rộng; trong một số thiết lập, một tác tử đơn còn tốt hơn nhiều tác tử phối hợp kém. Các vấn đề mở gồm quản trị thống nhất, ra quyết định tập thể, **lỗi lan truyền** giữa các tác tử, chi phí tài nguyên và thiếu benchmark chung. Bảng 3.1 dùng năm chiều để mô tả hai hệ được thực hiện.

Bảng 3.1. Ánh xạ khung năm chiều vào hai hệ đa tác tử của báo cáo.

Chiều	Thị trường hai cửa hàng (Ch. 4–7)	Hệ mua sắm (Ch. 8)
Tác nhân	Một tác tử Q-learning, một tác tử theo luật	Catalog, Recommendation, Inventory, Coordinator
Kiểu quan hệ	Cạnh tranh lợi nhuận, không tổng bằng không	Hợp tác hoàn thành một yêu cầu mua
Chiến lược	Học từ phần thưởng (RL) đối đầu luật cố định	Phân vai; luật xác minh ngân sách, tồn kho
Cấu trúc	Không giao tiếp; tương tác qua môi trường	Tập trung qua Coordinator
Điều phối	Đồng thời: cả hai chọn rồi môi trường áp dụng	Tĩnh: tìm → lọc, xếp hạng → kiểm kho
Lỗi lan truyền	Phản ứng giảm giá dây chuyền của đối thủ	Tồn kho cũ dẫn tới xác nhận sai

Nhận xét: Hai hệ nằm ở hai đầu của khung: hệ thứ nhất cạnh tranh, không có thông điệp, mọi tương tác ẩn trong động lực thị trường; hệ thứ hai hợp tác, mọi tương tác là thông điệp tường minh. Vì vậy chỉ số đánh giá cũng khác hẳn: lợi nhuận cho hệ thứ nhất, độ đúng và chi phí phối hợp cho hệ thứ hai.

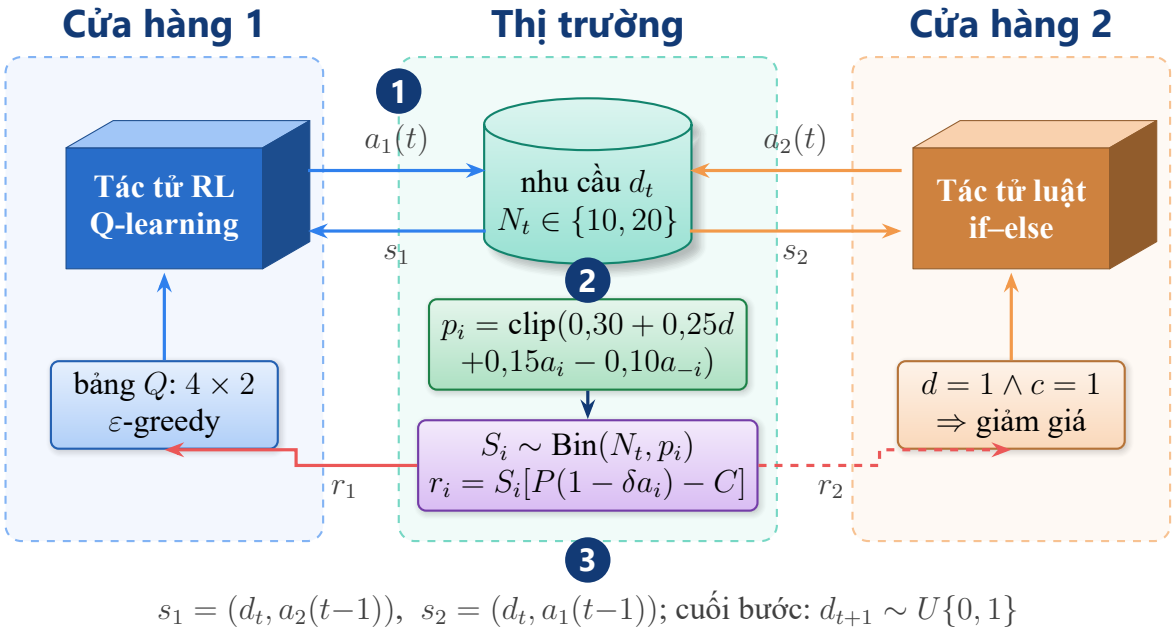
Chương 4

Mô hình thị trường và hai tác tử

Chương này hình thức hóa bài toán của Laboratory 02 [1]: môi trường, trạng thái, hành động, phần thưởng và hai chính sách. Trước khi chạy bất kỳ thí nghiệm nào, chương tính sẵn lợi nhuận kỳ vọng của mọi tổ hợp và chính sách tối ưu, để các kết quả học được ở Chương 6 có một mốc đối chiếu chính xác.

4.1 Mô tả bài toán

Hai cửa hàng thương mại điện tử cùng bán một loại sản phẩm. Ở mỗi bước thời gian t, mỗi cửa hàng $i \in \{1, 2\}$ chọn đồng thời một hành động $a_i(t) \in \{0, 1\}$: 0 là giữ giá gốc, 1 là giảm giá. Giảm giá kéo thêm khách nhưng làm mỏng biên lợi nhuận của từng đơn, và nó còn lấy bớt khách của đối thủ. Cửa hàng 1 dùng tác tử học tăng cường (Q-learning), cửa hàng 2 dùng tác tử theo luật cố định. Hình 4.1 tóm tắt một bước tương tác.



Hình 4.1. Vòng tương tác của hai tác tử trong một bước: (1) quan sát và chọn hành động đồng thời, (2) thị trường tính xác suất chuyển đổi, (3) lấy mẫu doanh số và trả thưởng.

Nhận xét: hai tác tử không trao đổi thông điệp; chúng ảnh hưởng nhau qua thị trường chung, đúng định nghĩa hệ đa tác tử ở Chương 1. Tác tử luật cũng nhận phần thưởng nhưng không dùng nó để đổi hành vi.

4.2 Trạng thái và hành động

Trạng thái cục bộ mà tác tử i quan sát trước khi hành động là

$$s_i(t) = \big(d_t,\ c_i(t)\big), \qquad d_t \in \{0,1\}, \quad c_i(t) = a_{-i}(t-1) \in \{0,1\}, \tag{4.1}$$

trong đó d_t là mức nhu cầu (0: thấp, 1: cao) và $c_i(t)$ là hành động của đối thủ ở bước trước. Mỗi tác tử có $|\mathcal{S}| = 4$ trạng thái và $|\mathcal{A}| = 2$ hành động, nên bảng Q chỉ có 8 ô. Trạng thái chung của cả hệ là bộ ba $(d_t, a_1(t-1), a_2(t-1))$ với 8 giá trị; tác tử chỉ thấy một phần của nó, cụ thể là không thấy hành động trước của chính mình. Điểm này quan trọng ở Mục 4.8.

4.3 Chuyển đổi khách hàng, doanh số và lợi nhuận

Xác suất một khách tiềm năng mua hàng của cửa hàng i là

$$p_i(t) = \text{clip}\big(0{,}30 + 0{,}25\,d_t + 0{,}15\,a_i(t) - 0{,}10\,a_{-i}(t);\ 0{,}05;\ 0{,}95\big). \tag{4.2}$$

Số khách tiềm năng là $N_t = 10$ khi $d_t = 0$ và $N_t = 20$ khi $d_t = 1$; số đơn bán được $S_i(t) \sim \text{Binomial}(N_t, p_i(t))$. Với giá gốc $P = 10$, giá vốn $C = 5$ và mức giảm $\delta = 0{,}20$, phần thưởng là lợi nhuận

$$r_i(t) = S_i(t)\,\big[P\,(1-\delta\,a_i(t)) - C\big], \qquad \mathbb{E}[r_i \mid d, a_i, a_{-i}] = N_d\,p_i\,\big[P(1-\delta a_i) - C\big]. \tag{4.3}$$

Biên lợi nhuận mỗi đơn là $10 - 5 = 5$ khi giữ giá và $8 - 5 = 3$ khi giảm giá: giảm giá cắt 40% biên lợi nhuận để đổi lấy 15 điểm phần trăm xác suất chuyển đổi.

Đoạn mã 4.1. Xác suất chuyển đổi theo (4.2).

```python
# ag2/market.py
def conversion_probability(cfg, demand, own, rival):
    """p_i(t) = clip(0,30 + 0,25 d + 0,15 a_i - 0,10 a_-i, 0,05, 0,95)."""
    p = (cfg.base_conversion + cfg.demand_effect * demand
         + cfg.own_discount_effect * own - cfg.rival_discount_effect *
         ↪ rival)
    return min(max(p, cfg.p_min), cfg.p_max)
```

4.4 Động lực môi trường

Sau khi trả thưởng, môi trường sinh nhu cầu kế tiếp d_{t+1} độc lập với hành động, đều trên $\{0, 1\}$; hành động hiện tại trở thành "hành động trước" của bước sau. Mỗi episode có $T = 20$ bước; đầu episode nhu cầu được rút ngẫu nhiên và hành động trước của cả hai đặt bằng 0. Lớp `MarketConfig` còn có tham số `demand_persist` $= P(d_{t+1} = d_t)$: giá trị $0{,}5$ cho đúng mô hình đề bài, giá trị $0{,}8$ cho biến thể "nhu cầu bền" dùng ở câu 9. Mã của lớp môi trường được trình bày ở Chương 5.

4.5 Tác tử Q-learning

Tác tử RL giữ bảng $Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. Sau mỗi bước nó cập nhật cặp vừa dùng [8]:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \Big[\underbrace{r_t + \gamma \max_{a'} Q(s_{t+1}, a')}_{\text{mục tiêu TD}} - Q(s_t, a_t) \Big], \tag{4.4}$$

với $\alpha = 0{,}10$, $\gamma = 0{,}90$; ở bước cuối episode mục tiêu chỉ còn r_t. Hành động được chọn theo ε-greedy: ngẫu nhiên với xác suất ε, ngược lại chọn $\arg\max_a Q(s, a)$ (hòa thì chọn ngẫu nhiên), với lịch giảm dần

$$\varepsilon_e = \max\big(\varepsilon_{\min},\ \varepsilon_0 \, \rho^{\,e}\big) = \max\big(0{,}02;\ 0{,}20 \cdot 0{,}995^{\,e}\big) \tag{4.5}$$

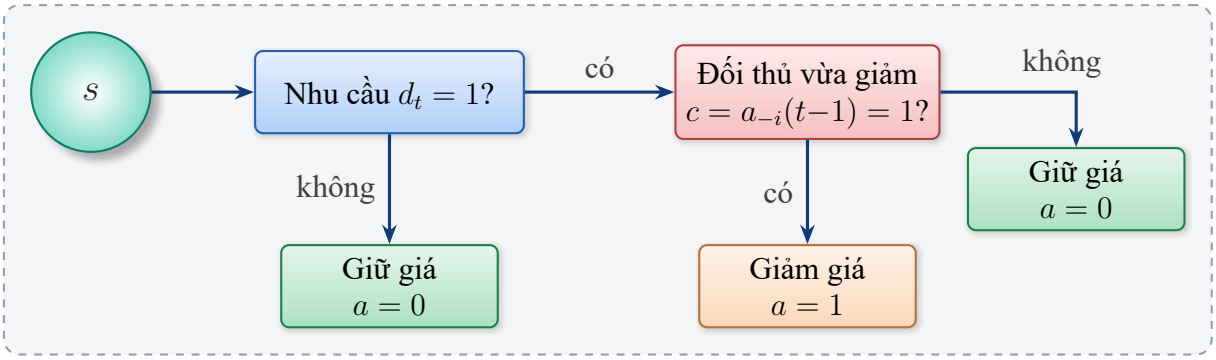
ở episode e. Sau khoảng 460 episode ε chạm sàn $0{,}02$ và giữ nguyên tới cuối.

4.6 Tác tử theo luật

Tác tử luật dùng chính sách if–else viết sẵn (Hình 4.2):

$$\pi_{\text{luật}}(d, c) = \begin{cases} 0, & d = 0, \\ 1, & d = 1 \text{ và } c = 1, \\ 0, & d = 1 \text{ và } c = 0. \end{cases} \tag{4.6}$$

Nhu cầu thấp thì giữ giá; nhu cầu cao thì chỉ giảm giá để "trả đũa" khi đối thủ vừa giảm.



Chuỗi chính sách theo thứ tự (0,0), (0,1), (1,0), (1,1): `0001`

Hình 4.2. Cây quyết định của tác tử theo luật.

4.7 Phân tích lợi nhuận kỳ vọng một bước

Vì mô hình nhỏ, có thể tính chính xác (4.3) cho mọi tổ hợp nhu cầu d, hành động của mình và hành động *hiện tại* b của đối thủ (Bảng 4.1).

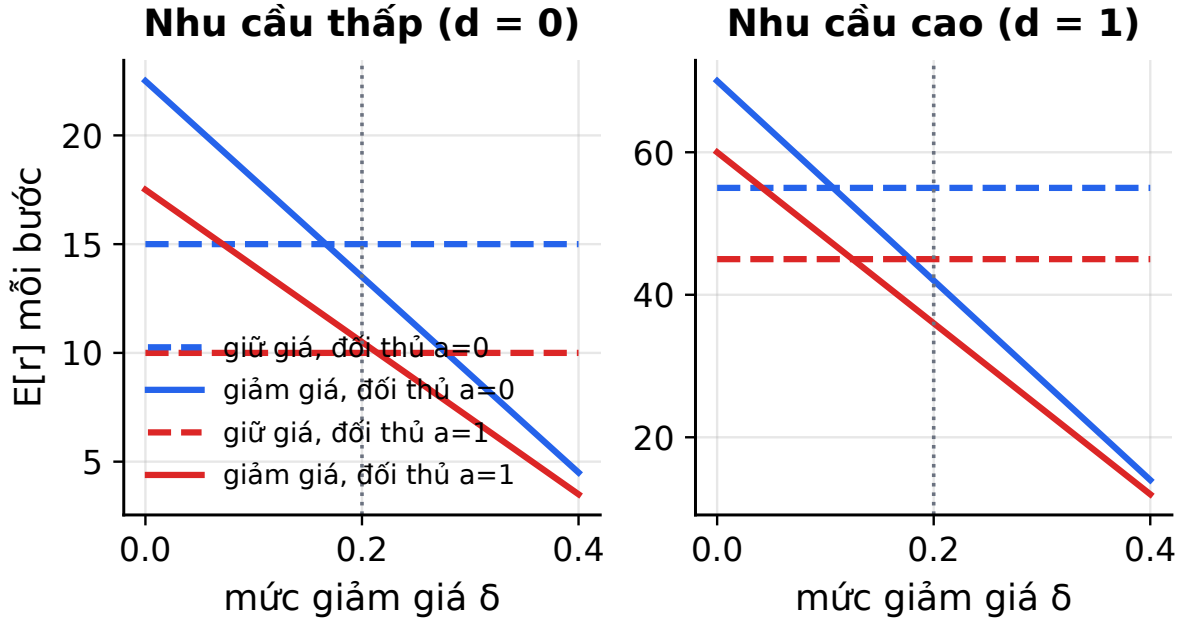
Nhận xét: với $\delta = 20\%$, giữ giá tốt hơn ở ba trên bốn ô. Chỉ ở ô $(0, 1)$, khi nhu cầu thấp và đối thủ đang giảm giá, giảm giá nhỉnh hơn ($10{,}5$ so với 10). Với $\delta = 5\%$ biên lợi nhuận khi giảm còn $4{,}5$, và giảm giá thắng ở mọi ô có nhu cầu thấp.

Hình 4.3 quét δ từ 0 đến 40%: đường liền (giảm giá) cắt đường đứt (giữ giá) tại các ngưỡng khác nhau theo từng trạng thái. Đây là lý do Thí nghiệm F (Chương 6) đổi δ để xem kết luận "RL hay luật tốt hơn" có đổi chiều hay không.

Bảng 4.1. Lợi nhuận kỳ vọng một bước của cửa hàng i theo (d, b), với $\delta = 20\%$ và $\delta = 5\%$.

(d, b)	p giữ	p giảm	E[r] giữ	E[r] giảm (δ = 20%)	Tốt hơn	E[r] giảm (δ = 5%)	Tốt hơn
(0, 0)	0,30	0,45	15,00	13,50	giữ	20,25	giảm
(0, 1)	0,20	0,35	10,00	10,50	giảm	15,75	giảm
(1, 0)	0,55	0,70	55,00	42,00	giữ	63,00	giảm
(1, 1)	0,45	0,60	45,00	36,00	giữ	54,00	giảm

b là hành động hiện tại của đối thủ; $\mathbb{E}[r] = N_d\, p \times$ biên lợi nhuận (output của scripts/summarize.py).



Hình 4.3. Lợi nhuận kỳ vọng một bước theo mức giảm giá δ; đường chấm đứng là $\delta = 20\%$.

4.8 Chính sách tối ưu khi đối thủ là tác tử luật

Bảng một bước chưa đủ, vì hành động hôm nay đổi trạng thái của đối thủ ngày mai. Nếu RL giảm giá ở bước t và nhu cầu ở $t + 1$ cao, tác tử luật sẽ giảm giá ở $t + 1$ và lợi nhuận của RL tụt từ 55 xuống 45. Để có mốc chính xác, hàm `exact_value` lan truyền phân phối của trạng thái chung qua 20 bước và cộng phần thưởng kỳ vọng, không lấy mẫu; `best_stationary` liệt kê toàn bộ $2^4 = 16$ chính sách tất định trên (d, c); `optimal_value` giải quy hoạch động ngược trên trạng thái chung và bước t (cận trên thật).

Kết quả phân tích (output của `scripts/summarize.py`)

- Hai tác tử luật đối đầu: không ai giảm giá (hành động trước ban đầu bằng 0), mỗi bên đạt 35 mỗi bước.
- Tối ưu trên 16 chính sách dừng: `0000` (không bao giờ giảm giá), giá trị 35; tối ưu quy hoạch động: 35. Tác tử luật vì vậy đã tối ưu ở cấu hình tham chiếu; RL giỏi nhất cũng chỉ hòa.
- Với $\delta = 5\%$: tối ưu là `1111` (luôn giảm giá), giá trị 37,35, cao hơn luật (35). Ở đây RL có dư địa để thắng.

Vì sao đây là bài toán đa tác tử khó

Hành động của đối thủ có mặt trong xác suất chuyển đổi, trong trạng thái và gián tiếp trong phần thưởng. Trạng thái (d, c) thiếu $a_i(t-1)$, thứ quyết định hành động *hiện tại* của tác tử luật, nên quá trình RL thấy không còn Markov hoàn toàn; khi đối thủ cũng học, môi trường còn trở thành phi dừng [10].

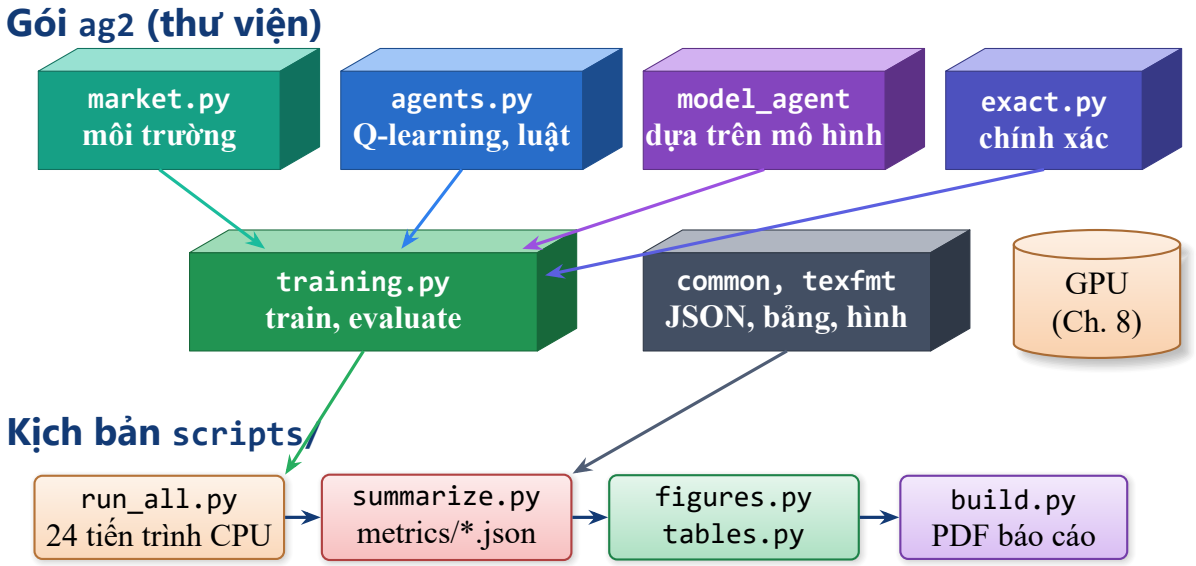
Chương 5

Cài đặt và thiết kế thí nghiệm

Chương này mô tả chương trình Python hiện thực mô hình của Chương 4, các thay đổi so với mã tham chiếu [11], giao thức đánh giá và danh mục thí nghiệm. Mọi đoạn mã dưới đây được `build.py` trích nguyên văn từ tệp nguồn đã chạy ra số liệu, nên mã trong báo cáo và mã thực thi luôn trùng nhau.

5.1 Kiến trúc chương trình

Chương trình tách thành thư viện `ag2` (mô hình, tác tử, vòng lặp, đánh giá chính xác) và các kịch bản `scripts/` (chạy thí nghiệm, tổng hợp, vẽ hình, sinh bảng). Hình 5.1 cho thấy luồng phụ thuộc: kịch bản chỉ gọi thư viện, kết quả đi qua JSON rồi mới vào báo cáo.



Hình 5.1. Kiến trúc chương trình: tầng thư viện `ag2` và tầng kịch bản; mũi tên là quan hệ sử dụng.

5.2 Các thay đổi so với mã tham chiếu

Đề bài cho phép sửa mã nếu ghi lại thay đổi và giữ so sánh hợp lệ, tái lập được [1]. Mô hình thị trường và mọi tham số giữ nguyên; các thay đổi chỉ nằm ở cách tổ chức và đo:

M1. **Số ngẫu nhiên chung.** Nhu cầu và doanh số của từng cửa hàng có bộ sinh riêng, tách từ một seed bằng `SeedSequence.spawn(3)`. Hai lượt đánh giá cùng seed thấy đúng một chuỗi nhu cầu; nếu hai chính sách chọn cùng hành động thì doanh số cũng giống hệt. Phép so sánh RL–luật vì vậy là so cặp (paired).

M2. **Giao diện tác tử thống nhất** `observe / act / learn`: một vòng lặp dùng chung cho RL–luật, RL–RL, các đối thủ cố định và tác tử mới ở câu 10.

M3. **Đánh giá chính xác và tối ưu** (`exact.py`, Mục 4.8): đo giá trị thật của chính sách tham lam, tách khỏi nhiễu Binomial.

M4. **Chạy song song** 291 lượt huấn luyện trên 24 tiến trình CPU.

M5. **Mở rộng** phục vụ câu 8–10: tham số `demand_persist`, `reward_kind`, Q-learning có trạng thái mở rộng, tác tử dựa trên mô hình, bốn đối thủ cố định.

5.3 Tham số tham chiếu

Seed của tác tử RL bằng seed môi trường cộng 10.000; seed đánh giá bằng seed huấn luyện cộng 500.000, nên không episode đánh giá nào trùng episode huấn luyện. Mười seed $7, 17, \ldots, 97$ là của riêng báo cáo này (nhóm 07).

Bảng 5.1. Tham số mô phỏng và huấn luyện (đọc trực tiếp từ `MarketConfig`, `QLearningAgent` và `run_all.py`).

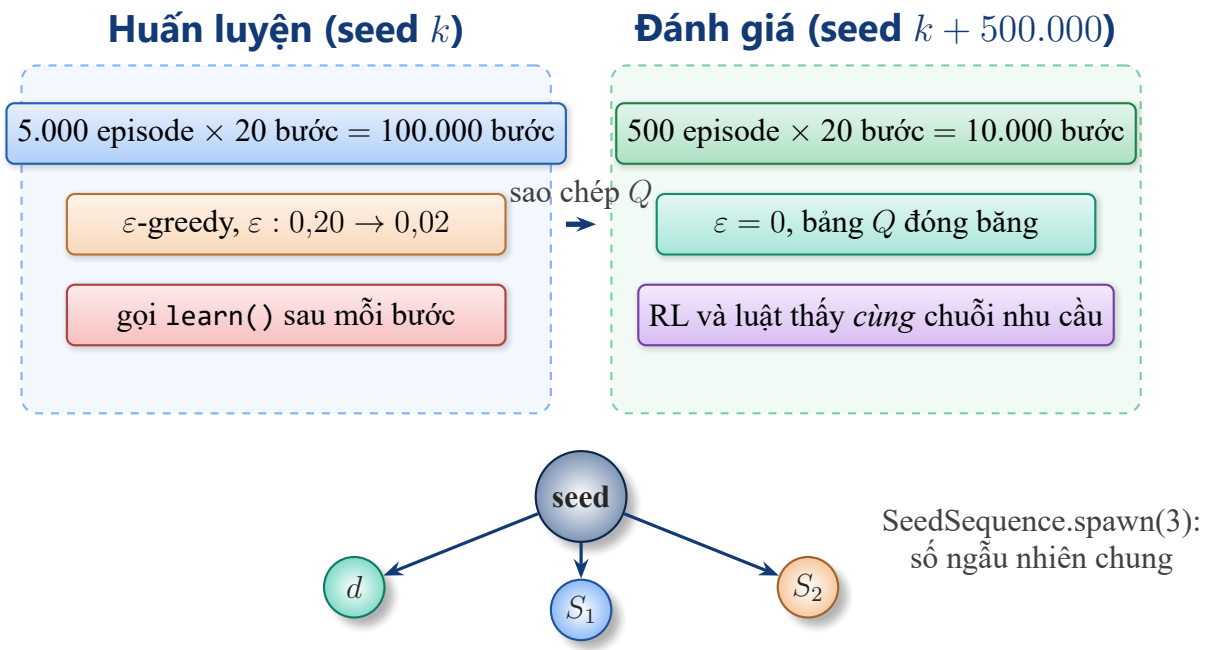
Tham số	Giá trị	Trong mã
Giá gốc P	10	`base_price`
Giá vốn C	5	`unit_cost`
Mức giảm δ	0,2	`discount_rate`
Khách khi d = 0	10	`customers_low`
Khách khi d = 1	20	`customers_high`
Chuyển đổi nền	0,3	`base_conversion`
Hiệu ứng nhu cầu	0,25	`demand_effect`
Hiệu ứng tự giảm giá	0,15	`own_discount_effect`
Hiệu ứng đối thủ giảm giá	0,1	`rival_discount_effect`
Số bước mỗi episode	20	`steps`
P(d giữ nguyên)	0,5	`demand_persist`
Tốc độ học α	0,1	`alpha`
Hệ số chiết khấu γ	0,9	`gamma`
ε_0 / $\varepsilon_{\min}$ / ρ	0,2 / 0,02 / 0,995	`eps_*`
Episode huấn luyện / đánh giá	5.000 / 500	`run_all`
Seed tham chiếu / đánh giá	2026 / 502026	`BASE_SEED`
Mười seed (C–H, câu 10)	7, 17, 27, 37, 47, 57, 67, 77, 87, 97	`SEEDS`

5.4 Giao thức đánh giá

Phần thưởng đo trong lúc huấn luyện trộn hai thứ: chất lượng chính sách và cái giá của khám phá. Báo cáo vì vậy đánh giá tách biệt (Hình 5.2): sau huấn luyện, bảng Q đóng băng, $\varepsilon = 0$, không gọi `learn()`, chạy 500 episode trên môi trường mới với seed riêng. Tác tử luật được đặt vào đúng ghế của RL và chạy lại trên cùng seed để có phép so cặp.

Đoạn mã 5.1. Một episode dùng chung cho huấn luyện và đánh giá.

```python
# ag2/training.py
def run_episode(env, agents, explore, learn, episode=0, on_update=None):
    """Một episode 20 bước; trả mảng phần thưởng (T, 2) và hành động (T,
    ↪ 2)."""
    for ag in agents:
        if hasattr(ag, "start_episode"):
            ag.start_episode(episode)
    joint = env.reset()
    rewards, actions = [], []
```



Hình 5.2. Giao thức huấn luyện – đánh giá tách biệt và ba bộ sinh số ngẫu nhiên chung.

```
    done = False
    while not done:
        obs = [ag.observe(joint, i) for i, ag in enumerate(agents)]
        acts = [ag.act(o, explore=explore) for ag, o in zip(agents, obs)]
        r, _, done = env.step(acts)
        joint = env.joint_state()
        if learn:
            for i, ag in enumerate(agents):
                nxt = ag.observe(joint, i)
                td = ag.learn(obs[i], acts[i], r[i], nxt, done)
                if on_update is not None and i == 0:
                    on_update(obs[i], acts[i], r[i], nxt, done, td)
        rewards.append(r)
        actions.append(acts)
    return np.array(rewards, dtype=float), np.array(actions, dtype=int)
```

5.5 Danh mục thí nghiệm và chỉ số đo

Các chỉ số dùng nhất quán: tổng thưởng và thưởng trung bình mỗi bước $\bar{R}_i = \frac{1}{T}\sum_{t=1}^{T} r_i(t)$ với $T = 10.000$ bước đánh giá; tỉ lệ giảm giá $\rho_i = \#\{a_i = 1\}/T$ (chỉ mô tả hành vi, không đồng nghĩa lợi nhuận); giá trị chính xác V^{π} của chính

Bảng 5.2. Tám thí nghiệm; A–E theo đề bài, F–H và câu 10 là phần mở rộng.

Mã	Nội dung	Cấu hình	Mục đích
A	Baseline	$\alpha = 0{,}10$, seed 2026	Hiệu năng cơ sở, bảng Q
B	Quá trình học	lịch sử của A, Q từng episode	Học có cải thiện không
C	Tốc độ học	$\alpha \in \{0{,}01;\ 0{,}10;\ 0{,}50\}$, 10 seed	Độ ổn định, độ nhạy
D	Khám phá	4 lịch (Bảng 5.3), 10 seed	Khám phá – khai thác
E	So sánh công bằng	10 seed, so cặp, KTC 95%	Độ tin cậy của kết luận
F	Mức giảm giá	$\delta \in \{5, 10, \ldots, 30\}\%$, 10 seed	Thiết kế thưởng (câu 5, 9)
G	Môi trường khác	nhu cầu bền $0{,}8$; thưởng = doanh thu	Độ bền (câu 5, 9)
H	Hai tác tử cùng học	Q-learning đối Q-learning	Phi dừng (câu 8)

Bảng 5.3. Bốn lịch khám phá của Thí nghiệm D, $\varepsilon_e = \max(\varepsilon_{\min}, \varepsilon_0 \rho^e)$.

Lịch	ε_0	$\varepsilon_{\min}$	ρ
Thấp	0,05	0,01	0,995
Cơ sở	0,2	0,02	0,995
Cao	0,5	0,05	0,999
Không khám phá	0	0	1

sách tham lam và số seed đạt tối ưu; chênh lệch so cặp RL − luật với khoảng tin cậy 95% theo phân phối t (9 bậc tự do); số lần chính sách tham lam đổi trong huấn luyện ("đổi CS").

5.6 Trích đoạn mã nguồn chính

Đoạn mã 5.2. Chọn hành động ε-greedy.

```python
# ag2/agents.py
def act(self, obs, explore=True):
    if explore and self.rng.random() < self.epsilon:
        return int(self.rng.integers(2))
    return self.greedy(obs)
```

Đoạn mã 5.3. Cập nhật Q-learning theo (4.4).

```python
# ag2/agents.py
def learn(self, obs, action, reward, next_obs, done):
    """Q(s,a) ← Q(s,a) + α[r + γ·max_a' Q(s',a') − Q(s,a)]; ở bước cuối mục tiêu chỉ còn r."""
    target = reward if done else reward + self.gamma * self.q[next_obs].max()
    td_error = target - self.q[obs][action]
    self.q[obs][action] += self.alpha * td_error
    self.visits[obs][action] += 1
    return td_error
```

5.7 Môi trường chạy và phân chia CPU/GPU

Máy chạy có 32 luồng CPU và một GPU NVIDIA RTX 4060 Laptop (8 GB); phần mềm Python 3.13.5, NumPy 2.5.1, Matplotlib 3.11.1. Một lượt Q-learning (100.000 bước) mất trung bình 4,761 s, một lượt tác tử dựa trên mô hình (quy hoạch lại mỗi episode) mất 30,73 s. Nếu chạy tuần tự, toàn bộ cần 2.837 s; chạy song song trên 24 tiến trình hết 143,3 s.

Vì sao Q-learning không chạy trên GPU

Bảng Q chỉ có 8 ô và mỗi bước phụ thuộc bước trước, nên không có phép tính lớn nào để song song hóa trong một lượt; đưa lên GPU chỉ thêm chi phí chép dữ liệu. Song song có lợi ở mức *lượt* (seed $\times$ cấu hình), là việc của CPU nhiều nhân. GPU được dùng ở Chương 8 cho bộ phân loại ý định PyTorch, chạy cùng lúc với các lượt CPU.

Tái lập toàn bộ: `python scripts/run_all.py`, rồi `summarize.py`, `figures.py`, `tables.py`, `ch8_run.py`, và `python build.py` trong thư mục báo cáo. Hai lượt chạy liên tiếp cho cùng số liệu đánh giá (thời gian chạy thì khác).

Chương 6

Kết quả thực nghiệm

Chương này trình bày kết quả của năm thí nghiệm A–E theo đề bài [1] và ba thí nghiệm mở rộng F–H phục vụ câu 8 và câu 9. Mọi con số được đọc từ `outputs/metrics/*.json` do `scripts/summarize.py` ghi. Với mỗi chính sách, báo cáo nêu hai đại lượng: *thưởng đánh giá* (lấy mẫu, 500 episode $\times$ 20 bước, $\varepsilon = 0$, bảng Q đóng băng, seed riêng) và *giá trị chính xác* (lan truyền phân phối trạng thái, không có nhiễu Binomial). Giá trị tối ưu khi đối thủ là tác tử luật ở cấu hình tham chiếu là 35 mỗi bước (Chương 4).

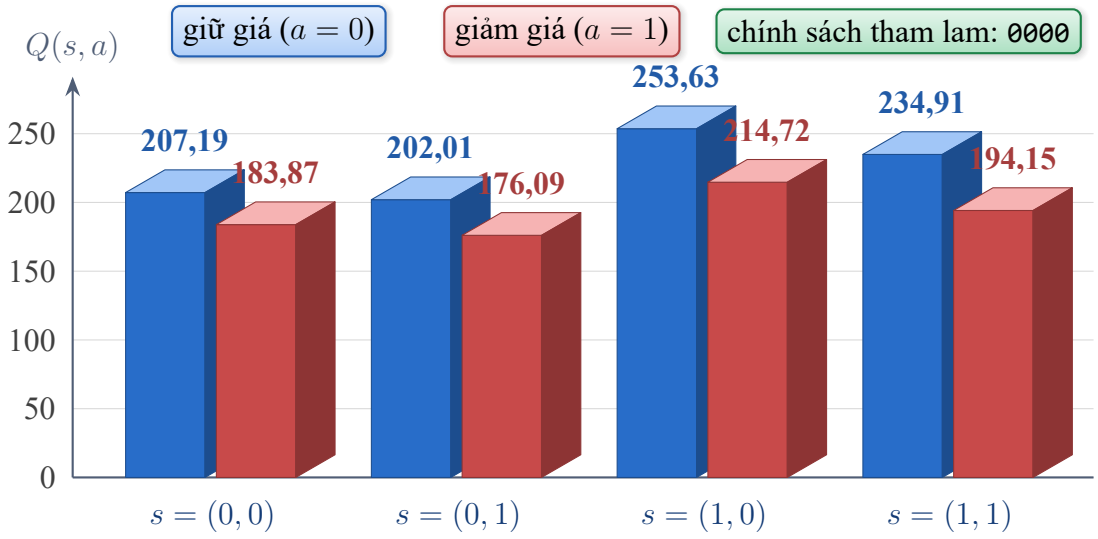
6.1 Thí nghiệm A: cấu hình tham chiếu

Lượt tham chiếu dùng seed tham chiếu của Bảng 5.1 cho huấn luyện; đánh giá chạy 10.000 bước trên môi trường mới. Tác tử Q-learning đạt tổng thưởng 347.985, tức 34,8 mỗi bước, tỉ lệ giảm giá 0,00%. Đặt tác tử luật vào cùng ghế, trên cùng chuỗi nhu cầu và cùng luồng doanh số, cho tổng 347.985: chênh lệch đúng bằng 0. Kết quả trùng khít này là do số ngẫu nhiên chung: hai chính sách cùng chọn "giữ giá" ở mọi trạng thái gặp phải nên nhận đúng cùng doanh số.

Bảng 6.1. Bảng Q sau 5.000 episode của lượt tham chiếu và số lần cập nhật từng cặp trạng thái–hành động.

s = (d, c)	Q(s, giữ)	Q(s, giảm)	lượt giữ	lượt giảm	Tham lam
(0, 0)	207,19	183,87	37.545	9.378	giữ
(0, 1)	202,01	176,09	1.986	1.189	giữ
(1, 0)	253,63	214,72	43.008	3.614	giữ
(1, 1)	234,91	194,15	3.173	107	giữ

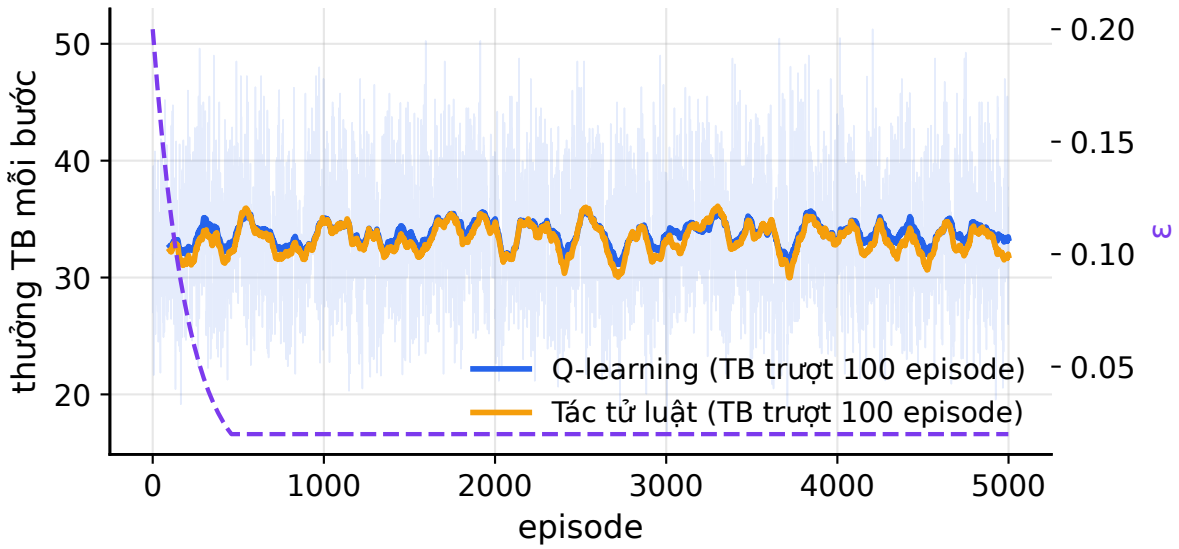
Nhận xét: Chính sách tham lam là `0000` (không bao giờ giảm giá), có giá trị chính xác 35 – đúng bằng mức tối ưu. Giá trị Q nhỏ hơn $\bar{r}/(1-\gamma) = 350$ vì episode kết thúc sau 20 bước mà quan sát không chứa bước t: mỗi ô Q là trung bình của lợi nhuận chiết khấu còn lại trên mọi vị trí trong episode. Số lượt cập



Hình 6.1. Bảng Q tham chiếu dưới dạng cột 3D: ở cả bốn trạng thái, cột "giữ giá" cao hơn cột "giảm giá".

nhật lệch mạnh: trạng thái $(1,1)$ chỉ có 107 lần thử giảm giá vì tác tử luật chỉ giảm giá khi tác tử RL vừa giảm giá; ước lượng ở các ô hiếm vì vậy nhiễu hơn hẳn.

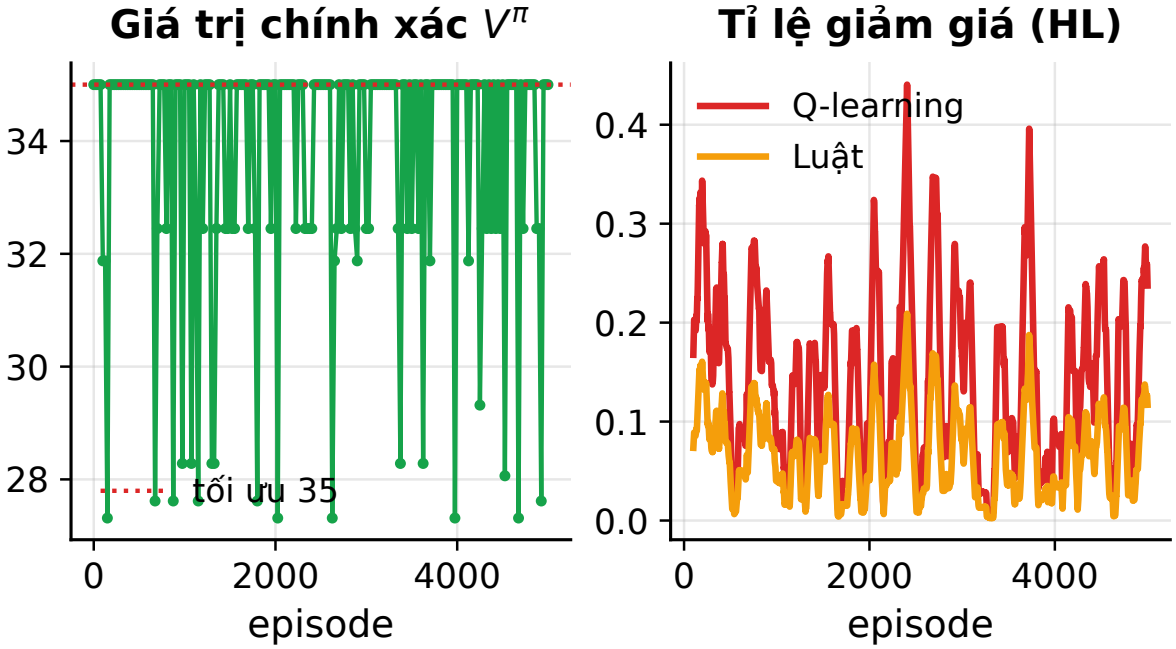
6.2 Thí nghiệm B: quá trình học



Hình 6.2. Thưởng mỗi bước theo episode khi huấn luyện (đường mờ: từng episode; đường đậm: trung bình trượt 100 episode) và lịch ε.

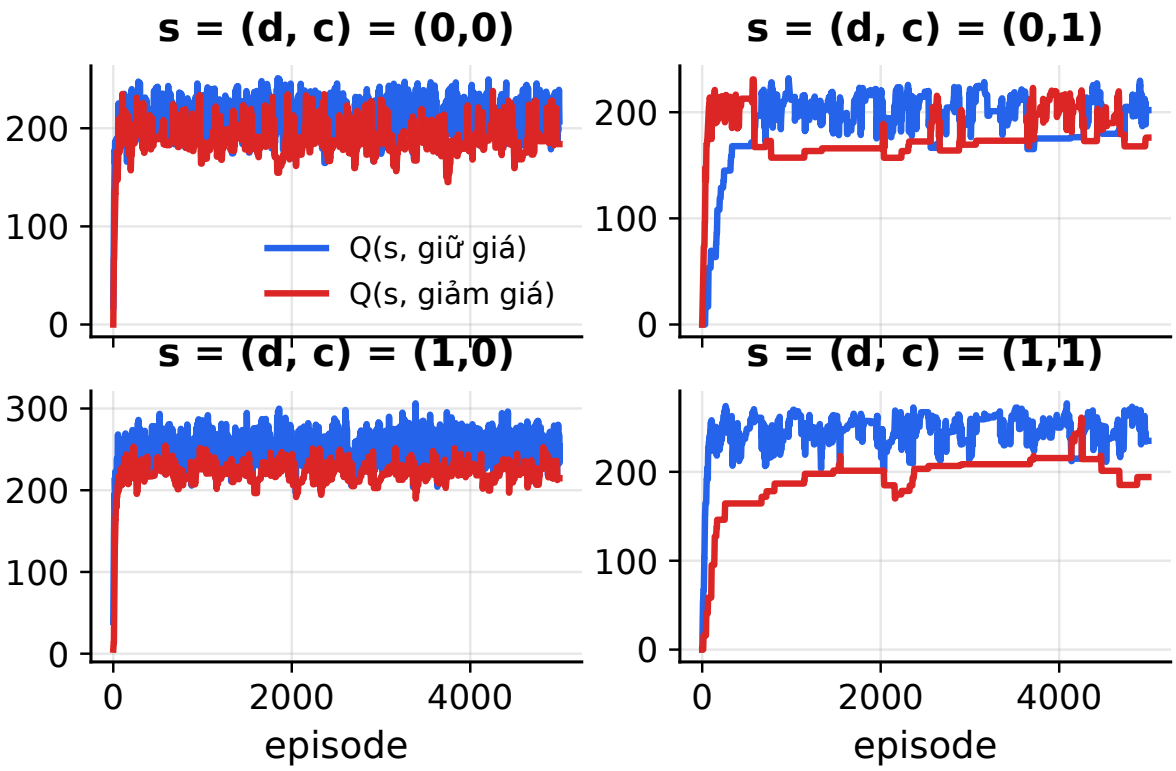
Nhận xét: Thưởng huấn luyện trung bình tăng từ 33,36 (500 episode đầu) lên

33,84 (500 episode cuối), còn tỉ lệ giảm giá giảm từ 19,32% xuống 13,23%. Đường cong rất nhiễu vì nhiễu Binomial lớn hơn nhiều so với mức cải thiện do học.



Hình 6.3. Trái: giá trị chính xác của chính sách tham lam, tính lại sau mỗi 25 episode. Phải: tỉ lệ giảm giá của hai tác tử khi huấn luyện.

Nhận xét: Giá trị chính xác đạt mức tối ưu sớm nhưng thỉnh thoảng rơi xuống: chính sách tham lam đổi qua 13 dạng khác nhau và chỉ ổn định từ episode 4.973. Nguyên nhân thấy rõ ở Hình 6.4: ở trạng thái $(0, 1)$, hai đường Q sát nhau (giảm giá chỉ kém một chút, Bảng 4.1) và trạng thái này hiếm, nên vài mẫu Binomial may mắn đủ đảo thứ tự. Phần lớn các lần đảo nằm ở trạng thái không bao giờ xuất hiện khi đánh giá với đối thủ luật, nên giá trị chính xác hầu như không đổi.



Hình 6.4. Quỹ đạo tám giá trị Q theo episode (lượt tham chiếu).

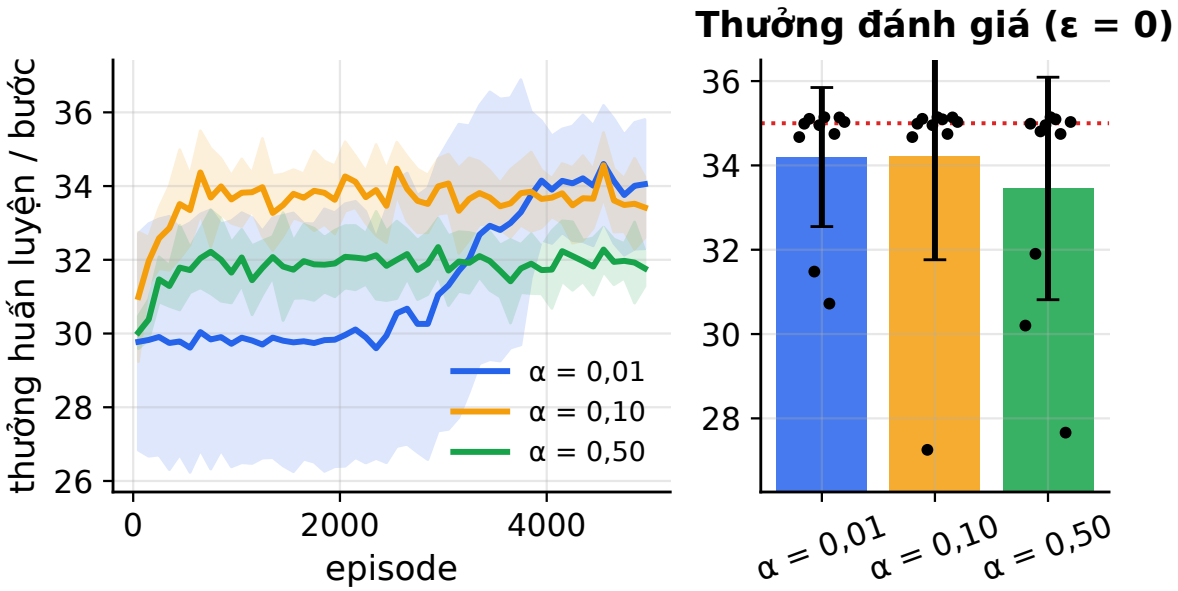
6.3 Thí nghiệm C: tốc độ học

Ba giá trị $\alpha \in \{0{,}01;\ 0{,}10;\ 0{,}50\}$ được chạy với 10 seed; các tham số khác giữ nguyên. Cột "Tối ưu" là số seed có chính sách cuối đạt giá trị chính xác tối ưu; "Đổi CS" là số lần chính sách tham lam thay đổi giữa hai episode liên tiếp.

Bảng 6.2. Ảnh hưởng của tốc độ học (10 seed, đánh giá tham lam 10.000 bước mỗi seed).

Cấu hình	Đánh giá TB ± SD	Chính xác	Tối ưu	Giảm giá	HL TB	Đổi CS	RL − luật (KTC 95%)
α = 0,01	34,20 ± 1,65	**34,25**	8/10	0,101	31,34	16	[−1,94; 0,40]
α = 0,10	34,21 ± 2,45	34,23	9/10	0,082	33,62	557	[−2,46; 0,95]
α = 0,50	33,45 ± 2,64	33,51	7/10	0,169	31,82	1.752	[−3,41; 0,38]

Nhận xét: $\alpha = 0{,}50$ cho thưởng đánh giá thấp nhất (33,45) và chính sách đổi 1.752 lần, gấp nhiều lần $\alpha = 0{,}10$ (557,2): mỗi cập nhật thay một nửa giá trị cũ bằng một mẫu nhiễu nên Q không bao giờ lắng. $\alpha = 0{,}01$ ổn định nhất (16,3 lần đổi) nhưng học chậm, thưởng huấn luyện chỉ 31,34. Cả ba cấu hình đều có



Hình 6.5. Thí nghiệm C: đường học (TB ± SD trên 10 seed, cửa sổ 100 episode) và thưởng đánh giá theo α (chấm đen: từng seed; nét đỏ: tối ưu).

khoảng tin cậy chênh lệch với luật chứa 0.

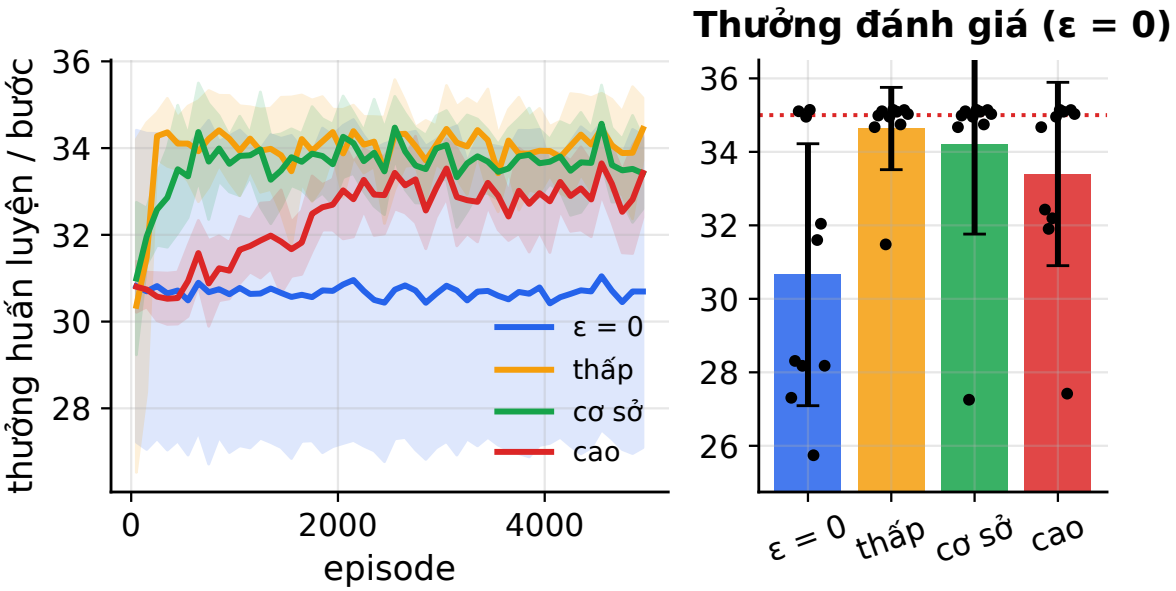
6.4 Thí nghiệm D: mức khám phá

Bốn lịch khám phá (Bảng 5.3) được so sánh; lịch "Không khám phá" ($\varepsilon = 0$) được thêm làm đối chứng cực đoan.

Bảng 6.3. Ảnh hưởng của khám phá (10 seed).

Cấu hình	Đánh giá TB ± SD	Chính xác	Tối ưu	Giảm giá	HL TB	Đổi CS	RL − luật (KTC 95%)
ε = 0	30,66 ± 3,56	30,67	3/10	0,382	30,68	0	[−6,82; −1,80]
Thấp	34,64 ± 1,12	**34,69**	9/10	0,050	33,95	298	[−1,08; 0,42]
Cơ sở	34,21 ± 2,45	34,23	9/10	0,082	33,62	557	[−2,46; 0,95]
Cao	33,40 ± 2,49	33,44	6/10	0,201	32,39	1.395	[−3,30; 0,16]

Nhận xét: Không khám phá là cấu hình tệ nhất: chỉ 3/10 seed tối ưu, thưởng đánh giá 30,66. Với Q khởi tạo bằng 0, hành động đầu tiên nhận thưởng dương sẽ được giữ mãi vì hành động còn lại không bao giờ được thử. Lịch thấp đạt tốt nhất (34,64, 9/10 tối ưu) vì bài toán chỉ có 8 ô Q; lịch cao giữ ε lớn lâu nên còn nhiều hành động giảm giá (20,11% khi đánh giá) và 6/10 tối ưu. Khám phá vừa



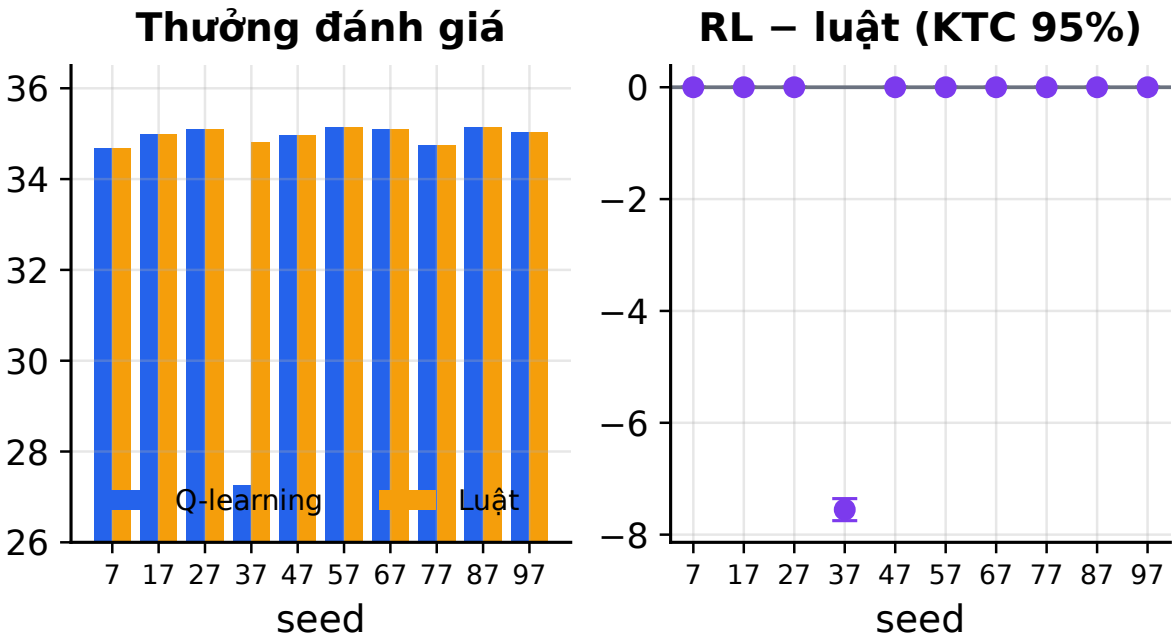
Hình 6.6. Thí nghiệm D: đường học theo lịch khám phá và thưởng đánh giá ($\varepsilon = 0$ khi đánh giá).

đủ là cần, khám phá nhiều chỉ tốn lợi nhuận trong huấn luyện.

6.5 Thí nghiệm E: so sánh công bằng trên nhiều seed

Mỗi seed huấn luyện một tác tử mới; đánh giá dùng seed huấn luyện $+500.000$. Tác tử luật được đánh giá trên đúng chuỗi nhu cầu đó (so cặp, mục 5.4).

Nhận xét: RL đạt 34,21 $\pm$ 2,45, luật đạt 34,97. Chênh lệch cặp trung bình $-0{,}7553$, KTC 95% qua 10 seed $[-2{,}464;\ 0{,}9532]$ chứa 0: không đủ bằng chứng để nói một bên tốt hơn. 9/10 seed đạt tối ưu; các seed có chính sách `0100` vẫn tối ưu vì trạng thái $(0, 1)$ không xuất hiện khi đối thủ là luật. Một seed rơi vào chính sách giảm giá ở nhiều trạng thái và kéo trung bình xuống: độ lệch chuẩn qua seed lớn chủ yếu do một lượt học hỏng, không do nhiễu đánh giá.



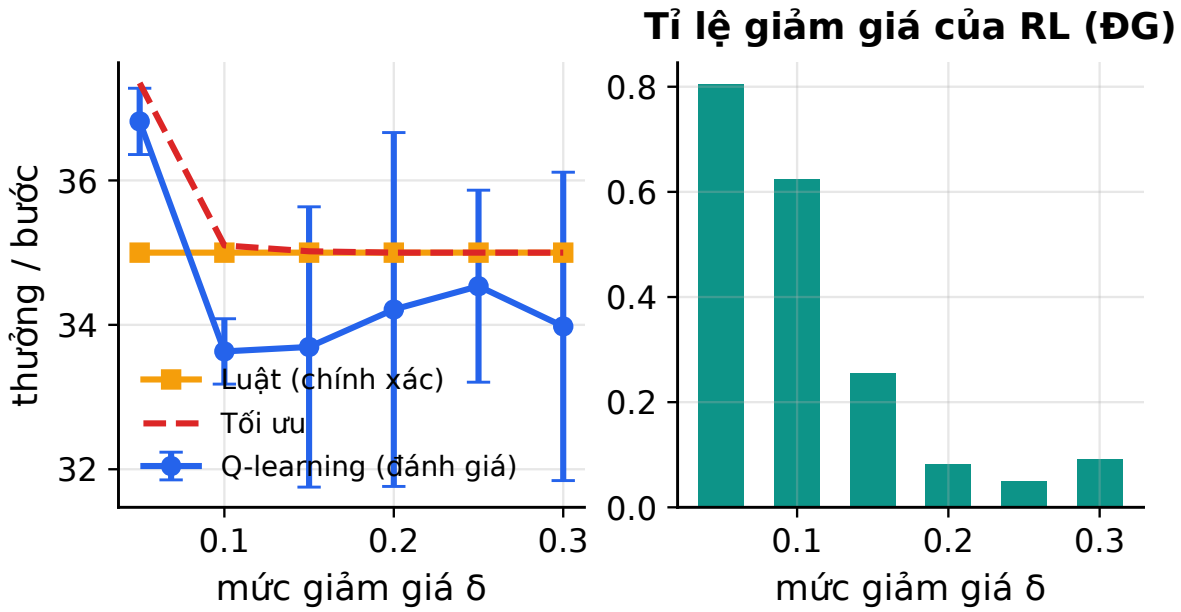
Hình 6.7. Thí nghiệm E: thưởng đánh giá theo seed và chênh lệch cặp RL − luật với khoảng tin cậy 95% trong từng seed.

Bảng 6.4. So sánh RL và luật trên 10 seed ("Chính xác": giá trị chính xác của chính sách RL; tối ưu = 35).

Seed	RL	Luật	RL − luật	Chính xác	Chính sách	Giảm giá
7	34,672	34,672	+0,000	35,00	`0000`	0,000
17	34,989	34,989	+0,000	35,00	`0000`	0,000
27	35,105	35,105	+0,000	35,00	`0001`	0,000
37	27,253	34,806	−7,553	27,31	`1110`	0,818
47	34,956	34,956	+0,000	35,00	`0100`	0,000
57	35,143	35,143	+0,000	35,00	`0100`	0,000
67	35,093	35,093	+0,000	35,00	`0100`	0,000
77	34,746	34,746	+0,000	35,00	`0000`	0,000
87	35,139	35,139	+0,000	35,00	`0100`	0,000
97	35,030	35,030	+0,000	35,00	`0000`	0,000
TB	34,21 ± 2,45	34,968	−0,755	34,23	9/10 tối ưu	0,082

6.6 Thí nghiệm F (mở rộng): mức giảm giá

Mức giảm δ đổi từ 5% đến 30%; luật giữ nguyên. Tối ưu được tính bằng quy hoạch động trên trạng thái chung.



Hình 6.8. Thí nghiệm F: thưởng của RL, của luật và mức tối ưu theo δ; phải: tỉ lệ giảm giá của RL khi đánh giá.

Bảng 6.5. Ảnh hưởng của mức giảm giá (10 seed mỗi mức).

Cấu hình	Đánh giá TB ± SD	Chính xác	Tối ưu	Giảm giá	HL TB	Đổi CS	RL − luật (KTC 95%)
δ = 5%	36,82 ± 0,46	**36,84**	2/10	0,805	36,64	660	[1,60; 2,10]
δ = 10%	33,63 ± 0,45	33,67	1/10	0,623	33,93	727	[−1,68; −1,00]
δ = 15%	33,69 ± 1,94	33,71	6/10	0,255	33,44	680	[−2,62; 0,07]
δ = 20%	34,21 ± 2,45	34,23	9/10	0,082	33,62	557	[−2,46; 0,95]
δ = 25%	34,54 ± 1,33	34,58	9/10	0,050	33,77	466	[−1,41; 0,55]
δ = 30%	33,98 ± 2,14	34,02	8/10	0,091	33,86	392	[−2,49; 0,51]

Nhận xét: Khi $\delta = 5\%$, giảm giá có lợi ở mọi trạng thái; luật cứng nhắc chỉ đạt 35 còn RL học giảm giá (80,53%) và đạt 36,82, vượt luật với KTC $[1{,}599;\ 2{,}101]$. Ngược lại ở $\delta = 10\%$, RL thua luật rõ rệt (KTC $[-1{,}676;\ -0{,}9959]$): giảm giá khi nhu cầu thấp có lợi tức thời, nhưng hành động đó khiến tác tử luật giảm giá ở bước sau, đúng lúc nhu cầu có thể cao. Quan sát (d, c) không chứa hành động

trước của chính tác tử nên hậu quả này "ẩn" với Q-learning; môi trường không còn Markov theo quan sát. Từ $\delta \geq 20\%$, giữ giá chiếm ưu thế và RL quay về gần tối ưu.

6.7 Thí nghiệm G (mở rộng): nhu cầu bền và thưởng theo doanh thu

Hai thay đổi môi trường cho câu 5 và câu 9: (i) nhu cầu bền, $P(d_{t+1} = d_t) = 0{,}8$ thay vì độc lập; (ii) huấn luyện với thưởng là doanh thu $S\,P(1 - \delta a)$ nhưng vẫn đánh giá bằng lợi nhuận.

Bảng 6.6. Hai biến thể môi trường (10 seed; đánh giá luôn theo lợi nhuận).

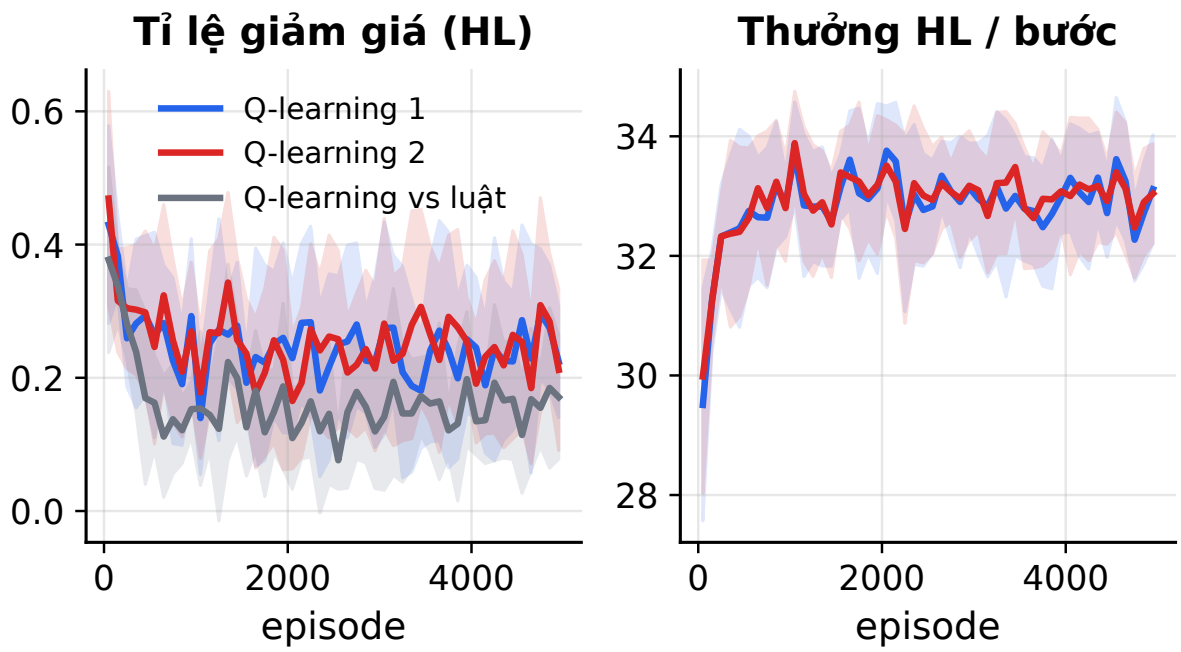
Cấu hình	Đánh giá TB ± SD	Chính xác	Tối ưu	Giảm giá	HL TB	Đổi CS	RL − luật (KTC 95%)
Nhu cầu bền	34,04 ± 1,92	**33,90**	6/10	0,180	33,42	554	[−2,48; 0,26]
Thưởng = doanh thu	28,19 ± 2,52	28,22	1/10	0,623	67,87	727	[−8,60; −4,95]

Nhận xét: Nhu cầu bền không đổi kết luận: tối ưu vẫn là giữ giá (35), RL đạt 34,04. Thưởng theo doanh thu thì làm hỏng hẳn mục tiêu: thưởng huấn luyện cao (67,87, tính bằng doanh thu) nhưng lợi nhuận khi đánh giá chỉ 28,19, regret 6,783, vì tác tử giảm giá 62,34% số bước để tăng số đơn. Tác tử tối ưu đúng cái được thưởng, không phải cái người thiết kế muốn.

6.8 Thí nghiệm H (mở rộng): hai tác tử Q-learning cùng học

Thay tác tử luật bằng một tác tử Q-learning thứ hai; cả hai cùng học từ đầu, đánh giá với cả hai đóng băng.

Nhận xét: Với đối thủ học, tỉ lệ giảm giá huấn luyện cao hơn (24,93% so với 16,43% khi đối thủ là luật) và thưởng đánh giá thấp hơn (33,19 và 32,44). Không seed nào cả hai cùng hội tụ về `0000` (0/10): chính sách cuối khác nhau theo seed, ví dụ seed 7 cho cặp `1000`–`1000`, seed 17 cho `1100`–`0000`; nhiều cặp chọn giảm giá khi nhu cầu thấp. Mỗi tác tử là một phần của môi trường đối với tác tử kia,



Hình 6.9. Thí nghiệm H: tỉ lệ giảm giá và thưởng khi huấn luyện của hai Q-learning cùng học (TB $\pm$ SD 10 seed); đường xám: Q-learning học với đối thủ luật.

nên phân phối phản hồi thay đổi trong suốt quá trình học – môi trường phi dừng, điều kiện hội tụ của Q-learning một tác tử không còn đúng.

6.9 Tổng hợp

Những điều kết quả cho thấy

- Ở cấu hình tham chiếu, luật đã tối ưu; RL chỉ hòa (tối ưu ở 9/10 seed, chênh lệch không có ý nghĩa thống kê).
- RL thắng khi giả định ẩn trong luật sai ($\delta = 5\%$), thua khi quan sát thiếu thông tin ($\delta = 10\%$) hoặc thưởng lệch mục tiêu; α lớn, khám phá dài hay hai tác tử cùng học đều làm chính sách dao động.

Chương 7

Trả lời mười câu hỏi của bài thí nghiệm

Mỗi câu trả lời dựa trên số liệu đo ở Chương 6 hoặc trên phép tính trực tiếp từ mô hình ở Chương 4; phần suy luận không có số đo đi kèm được nói rõ là giải thích.

7.1 Câu 1: Mô hình hóa tác tử và môi trường

Bảng 7.1. Các thành phần của hệ theo khung tác tử–môi trường.

Thành phần	Trong mô phỏng
Tác tử	Hai cửa hàng: tác tử Q-learning (ghế 1) và tác tử luật (ghế 2)
Môi trường	Thị trường: nhu cầu $d_t \in \{0, 1\}$, số khách N_d, xác suất chuyển đổi (4.2), doanh số Binomial
Trạng thái	$s_i(t) = (d_t,\ a_{-i}(t-1))$ – 4 trạng thái mỗi tác tử
Hành động	$a_i(t) \in \{0 : \text{giữ giá},\ 1 : \text{giảm giá}\}$, chọn đồng thời
Phần thưởng	Lợi nhuận $r_i = S_i\,[P(1-\delta a_i) - C]$ (4.3)
Chính sách	RL: tham lam theo bảng Q (8 giá trị); luật: (4.6)

Đây là hệ đa tác tử vì phần thưởng của mỗi cửa hàng phụ thuộc hành động của cửa hàng kia ở ba chỗ: số hạng $-0{,}10\,a_{-i}$ trong xác suất chuyển đổi, thành phần $a_{-i}(t-1)$ trong trạng thái, và phản ứng của đối thủ ở bước sau đối với hành động hôm nay. Hai tác tử không trao đổi thông điệp; chúng tương tác gián tiếp qua môi trường chung.

7.2 Câu 2: Quyết định dựa trên học và dựa trên luật

Ở cấu hình tham chiếu, luật đã trùng với chính sách tối ưu nên RL chỉ có thể hòa (34,21 so với 34,97, KTC chênh lệch chứa 0). Ưu thế của học chỉ lộ ra khi giả định ẩn trong luật sai: với $\delta = 5\%$, RL đạt 36,82 còn luật 35.

Bảng 7.2. So sánh hai cách ra quyết định.

Khía cạnh	Tác tử Q-learning	Tác tử luật
Nguồn tri thức	Kinh nghiệm: 100.000 bước tương tác	Người thiết kế viết sẵn
Thích nghi	Có: đổi chính sách khi δ đổi (5% $\rightarrow$ giảm giá)	Không: cùng hành vi ở mọi δ
Chi phí học	Khám phá làm mất lợi nhuận khi huấn luyện	Không có
Ổn định	Phụ thuộc seed, α, ε (Mục 6.5)	Tất định
Giải thích được	Phải đọc bảng Q	Đọc trực tiếp ba nhánh if–else

7.3 Câu 3: Tính một bước cập nhật Q-learning

Bước dưới đây lấy nguyên từ nhật ký huấn luyện của lượt tham chiếu (`first_updates` trong `outputs/raw/baseline.json`): tác tử ở $s = (00)$ chọn $a = 0$ (giữ giá), nhận $r = 25$ và chuyển sang $s' = (10)$. Trước cập nhật $Q(s,a) = 2{,}8$ và $\max_{a'} Q(s',a') = 4{,}252$. Theo (4.4) với $\alpha = 0{,}1$, $\gamma = 0{,}9$:

$$\begin{aligned} \text{mục tiêu} &= r + \gamma \max_{a'} Q(s',a') = 25 + 0{,}9 \times 4{,}252 = 28{,}83, \\ \text{sai số TD} &= 28{,}83 - 2{,}8 = 26{,}03, \\ Q_{\text{mới}}(s,a) &= 2{,}8 + 0{,}1 \times 26{,}03 = 5{,}403. \end{aligned}$$

Giá trị chương trình thực sự ghi vào bảng là 5,403, khớp phép tính tay. Với $\alpha = 0{,}1$, mỗi quan sát chỉ dịch Q đi 10% khoảng cách tới mục tiêu, nên một mẫu doanh số may mắn không làm đảo chính sách ngay.

7.4 Câu 4: Khám phá và khai thác

Thí nghiệm D (Bảng 6.3) cho câu trả lời định lượng. Không khám phá ($\varepsilon = 0$): chỉ 3/10 seed tối ưu, thưởng đánh giá 30,66, vì hành động thử đầu tiên có thưởng dương sẽ được khai thác mãi. Lịch thấp đạt 34,64 với 9/10 tối ưu; lịch cao còn 6/10 và giữ tỉ lệ giảm giá 20,11% khi đánh giá. Với 8 ô Q và 100.000 bước, cần rất ít khám phá; khám phá thêm là chi phí trả bằng lợi nhuận huấn luyện (32,39 so với 33,95). Trong môi trường lớn hơn hoặc thưởng thưa, cân bằng này dịch

về phía khám phá nhiều hơn – đây là giải thích, chưa được đo trong bài.

7.5 Câu 5: Phần thưởng và mục tiêu kinh doanh

Với $P = 10$, $C = 5$, $\delta = 0{,}2$: giữ giá lãi 5 mỗi đơn, giảm giá còn $8 - 5 = 3$, tức mất 40% biên lợi nhuận để đổi lấy 15 điểm phần trăm xác suất chuyển đổi. Giảm giá tăng doanh số nhưng chỉ tăng lợi nhuận khi $N\,p_1 \cdot 3 > N\,p_0 \cdot 5$, điều chỉ xảy ra ở trạng thái $(0, 1)$ (Bảng 4.1). Thí nghiệm G kiểm chứng tầm quan trọng của việc chọn thưởng: huấn luyện với thưởng là doanh thu cho thưởng huấn luyện 67,87 nhưng lợi nhuận đánh giá chỉ 28,19 (regret 6,783), vì tác tử giảm giá 62,34% số bước. Phần thưởng phải đo đúng mục tiêu kinh doanh; nếu muốn cân nhắc thêm thị phần hay giữ chân khách, phải đưa chúng vào thưởng một cách tường minh.

7.6 Câu 6: Đánh giá hiệu năng từ dữ liệu thực nghiệm

Lượt tham chiếu: $\bar{R}_{RL} = 347.985/10.000 = 34{,}8$, tỉ lệ giảm giá 0,00%; tác tử luật trên cùng luồng ngẫu nhiên cũng đạt 34,8. Một lượt không đủ để kết luận, nên Thí nghiệm E lặp 10 seed: RL 34,21 $\pm$ 2,45, chênh lệch cặp $-0{,}7553$, KTC 95% $[-2{,}464;\ 0{,}9532]$. Ba nguyên tắc rút ra: (i) đánh giá đóng băng với $\varepsilon = 0$, không dùng thưởng huấn luyện; (ii) so cặp trên cùng chuỗi nhu cầu để loại phương sai do mẫu; (iii) báo cáo phân bố qua seed và giá trị chính xác, vì độ lệch chuẩn 2,45 chủ yếu đến từ một seed học hỏng chứ không phải từ nhiễu đánh giá.

7.7 Câu 7: Tốc độ học và tính ổn định

Bảng 6.2: $\alpha = 0{,}50$ cho thưởng đánh giá 33,45 và 1.752 lần đổi chính sách; $\alpha = 0{,}10$ cho 34,21 và 557,2 lần; $\alpha = 0{,}01$ cho 34,2 và chỉ 16,3 lần. α lớn làm Q bám theo mẫu doanh số gần nhất – với $\alpha = 0{,}5$, mẫu mới chiếm một nửa giá trị – nên ở trạng thái gần hòa như $(0, 1)$ thứ tự hai hành động liên tục đảo. α nhỏ làm trung bình mượt nhưng học chậm (thưởng huấn luyện 31,34). Lựa chọn hợp lý là α vừa phải hoặc giảm dần theo số lần thăm $\alpha_n(s, a) = 1/n(s, a)$, đúng điều kiện hội tụ của Q-learning [8].

7.8 Câu 8: Tương tác đa tác tử và tính phi dừng

Khi đối thủ là luật, phản ứng của nó là một hàm cố định nên với tác tử RL môi trường là dừng (dù không Markov đầy đủ theo quan sát (d, c)). Khi đối thủ cũng học (Thí nghiệm H), chính sách của nó đổi trong suốt quá trình học, nên cùng một cặp (s, a) cho phân phối thưởng khác nhau theo thời gian. Số đo: tỉ lệ giảm giá huấn luyện 24,93% (so với 16,43% khi đối thủ là luật), thưởng đánh giá 33,19 và 32,44, và không seed nào cả hai hội tụ về `0000`. Kết quả phụ thuộc lẫn nhau giữa hai quá trình học là đặc trưng của học tăng cường đa tác tử [10]; cách xử lý thường gặp là đưa hành động của đối thủ vào mô hình (như tác tử ở Câu 10) hoặc học tập trung, thực thi phân tán.

7.9 Câu 9: Giới hạn và thay đổi môi trường đã thực hiện

Giới hạn chính: (i) không gian rất nhỏ – 4 trạng thái, 2 hành động, không tồn kho, không nhiều mức giá; (ii) quan sát (d, c) thiếu hành động trước của chính tác tử, nên hậu quả "giảm giá hôm nay $\rightarrow$ đối thủ giảm giá ngày mai" bị ẩn; (iii) nhu cầu độc lập với lịch sử; (iv) thưởng đo bằng lợi nhuận mô phỏng, không có dữ liệu thật.

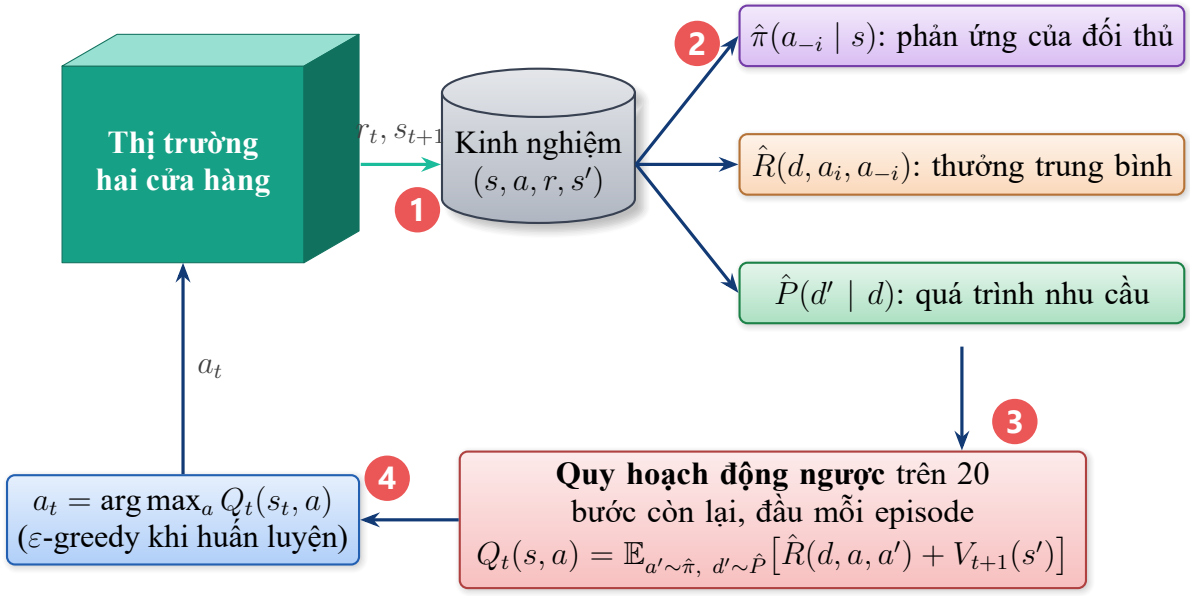
Hai thay đổi đã cài và đo. **Mức giảm giá** (Thí nghiệm F): ở $\delta = 10\%$ RL thua luật (KTC $[-1{,}676;\ -0{,}9959]$) chính vì giới hạn (ii): giảm giá khi nhu cầu thấp có lợi tức thời nhưng kích phản ứng của đối thủ. **Nhu cầu bền** $P(d_{t+1} = d_t) = 0{,}8$ (Thí nghiệm G): tối ưu vẫn là giữ giá (35); RL đạt 34,04 với 6/10 seed tối ưu, kém hơn môi trường gốc vì trạng thái nhu cầu cao kéo dài làm mẫu kém đa dạng. Kết luận "RL hòa luật" vì vậy chỉ đúng trong phạm vi mô hình tham chiếu.

7.10 Câu 10: Đề xuất và kiểm chứng tác tử tốt hơn

Hai tác tử được đề xuất và cài đặt, nhằm thẳng vào giới hạn (ii) ở Câu 9:

- **Q-learning mở rộng**: quan sát $(d,\ a_{-i}(t-1),\ a_i(t-1))$ – 8 trạng thái, 16 giá trị Q.
- **Tác tử dựa trên mô hình**: học từ kinh nghiệm phản ứng của đối thủ, bảng thưởng trung bình và quá trình nhu cầu, rồi lập kế hoạch bằng quy hoạch

động ngược trên đúng 20 bước còn lại (Hình 7.1). Nó tối ưu trực tiếp thước đo đánh giá (lợi nhuận không chiết khấu mỗi episode) thay vì mục tiêu chiết khấu $\gamma = 0{,}9$.



Hình 7.1. Kiến trúc tác tử dựa trên mô hình: (1) thu kinh nghiệm, (2) cập nhật ba mô hình đếm tần suất, (3) quy hoạch động ngược đầu mỗi episode, (4) chọn hành động tham lam theo Q_t.

Ba tác tử được huấn luyện 10 seed trên năm kịch bản: tham chiếu, $\delta = 5\%$, đối thủ ăn miếng trả miếng, đối thủ luôn giảm giá, nhu cầu bền. Bảng 7.3 báo giá trị chính xác trung bình; Hình 7.2 báo regret = tối ưu − giá trị chính xác.

Bảng 7.3. Giá trị chính xác mỗi bước (TB 10 seed) và số seed đạt tối ưu theo kịch bản.

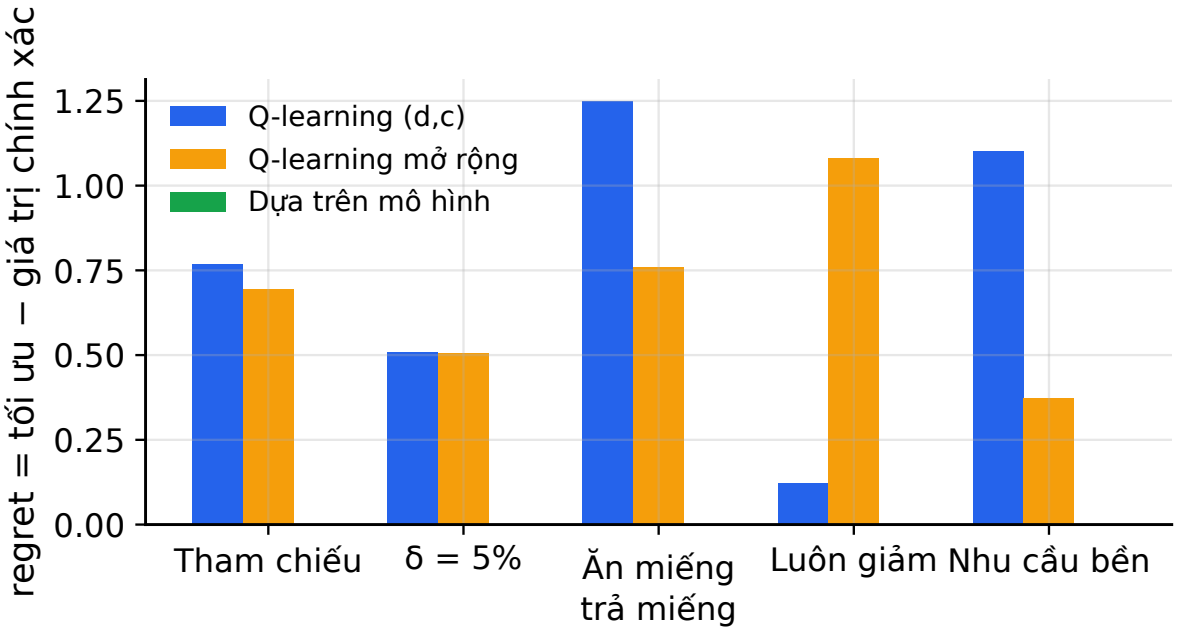
Kịch bản	Tối ưu	Q (d,c)	Q mở rộng	Mô hình	Luật	Tối ưu Q	Tối ưu Q+	Tối ưu MH
Tham chiếu (luật, δ = 20%)	35,00	34,23	34,30	**35,00**	35,00	9/10	7/10	10/10
Luật, δ = 5%	37,35	36,84	36,84	**37,35**	35,00	2/10	2/10	10/10
Ăn miếng trả miếng	35,00	33,75	34,24	**35,00**	35,00	7/10	8/10	10/10
Luôn giảm giá	27,75	27,63	26,67	**27,75**	23,23	3/10	0/10	10/10
Nhu cầu bền (0,8)	35,00	33,90	34,63	**35,00**	35,00	6/10	7/10	10/10

Nhận xét: Tác tử dựa trên mô hình đạt tối ưu ở cả năm kịch bản (10/10 seed ở tham chiếu, 10/10 ở $\delta = 5\%$ với giá trị 37,35 bằng tối ưu 37,35), trong khi Q-learning gốc chỉ đạt 2/10 seed ở $\delta = 5\%$. Q-learning mở rộng giảm regret khi

Đoạn mã 7.1. Quy hoạch động ngược của tác tử dựa trên mô hình.

```python
# ag2/model_agent.py
def backward_induction(self):
    """V_T = 0; Q_t(s,a) = Σ_a' π(a'|s) [R̂(d,a,a') + Σ_d' P̂(d'|d) V_{t+1}(d', a', a)]."""
    r_hat = self.reward_model()
    p_d = self.d_counts / self.d_counts.sum(axis=1, keepdims=True)
    v_next = {s: 0.0 for s in self.states}
    plan = []
    for _ in range(self.horizon):
        q_t, v_t = {}, {}
        for s in self.states:
            d = s[0]
            pi = self.opp_counts[s] / self.opp_counts[s].sum()
            q = np.zeros(2)
            for a in (0, 1):
                for b in (0, 1):
                    future = sum(p_d[d, d2] * v_next[(d2, b, a)] for d2 in (0, 1))
                    q[a] += pi[b] * (r_hat[d, a, b] + future)
            q_t[s], v_t[s] = q, q.max()
        plan.append(q_t)
        v_next = v_t
    return plan[::-1]
```

phản ứng của đối thủ phụ thuộc hành động trước của mình (ăn miếng trả miếng: 0,7598 so với 1,251; nhu cầu bền: 0,3718 so với 1,102), nhưng kém hơn khi đối thủ luôn giảm giá (1,082 so với 0,1212) vì gấp đôi số ô phải học với cùng lượng dữ liệu. Ưu điểm của tác tử dựa trên mô hình đến từ hai nguồn: dùng dữ liệu hiệu quả hơn (mỗi bước cập nhật mô hình dùng chung cho mọi kế hoạch) và lập kế hoạch đúng chân trời 20 bước. Đổi lại, nó cần biết cấu trúc trạng thái chung và giả định đối thủ dừng; trước một đối thủ cũng đang học, mô hình đếm tần suất sẽ cũ dần và phải thêm cơ chế quên.



Hình 7.2. Regret của ba tác tử trong năm kịch bản (TB 10 seed; thấp hơn là tốt hơn).

Chương 8

Hệ đa tác tử hỗ trợ mua sắm

Chương này hiện thực hệ mua sắm của Tutorial MAS [12] (tác tử ý định là mạng PyTorch trên GPU) để đo luận điểm của Chương 1: phân vai giúp xử lý lỗi, nhưng mỗi chặng thông điệp đều có giá về độ trễ.

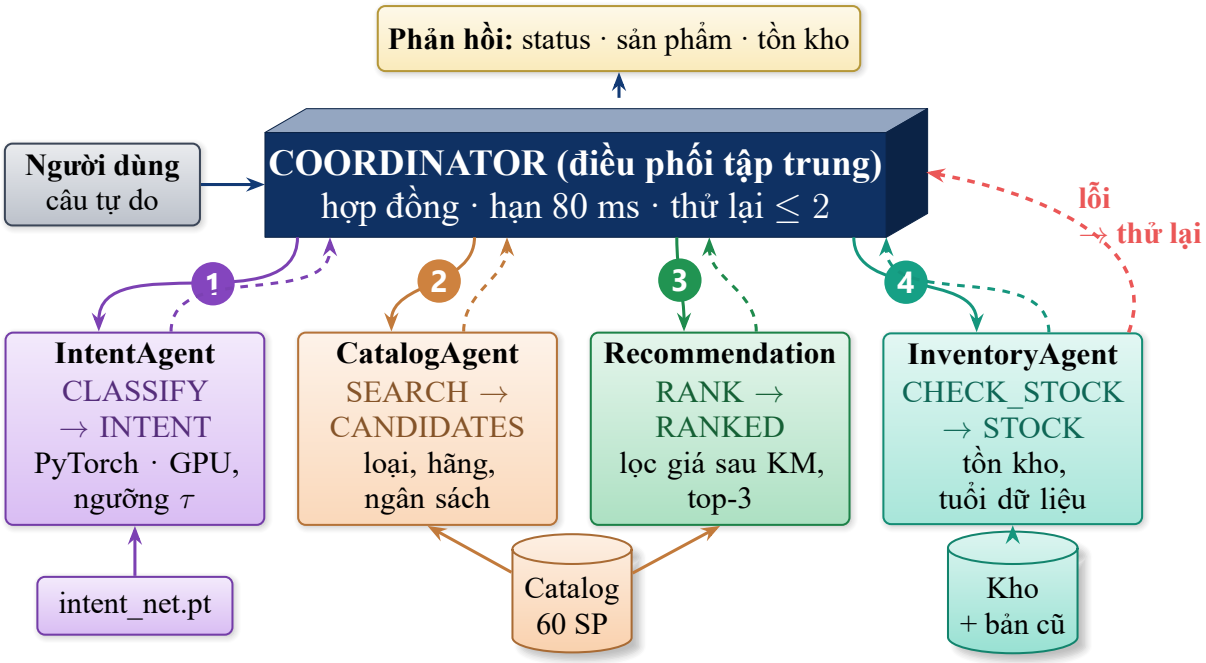
8.1 Đặc tả PEAS và luồng thông điệp

Catalog tổng hợp có 60 mặt hàng sáu loại; kho có 41 mặt hàng còn hàng. Môi trường (E) là phiên mua sắm; mọi tác tử chỉ *đọc* dữ liệu của mình (Bảng 8.1).

Bảng 8.1. Đặc tả PEAS của năm thành phần trong hệ mua sắm.

Tác tử	**Cảm biến (S) → Hành động (A), thông điệp**	**Hiệu năng (P)**
IntentAgent	câu người dùng → CLASSIFY/INTENT {ý định, độ tin cậy}; dưới ngưỡng τ thì hỏi lại	accuracy, macro-F1
CatalogAgent	câu, catalog → SEARCH/CANDIDATES {loại, hãng, ngân sách, id}; không nhận ra loại thì trả rỗng	đúng tập ứng viên
Recommendation	ứng viên → RANK/RANKED: lọc giá sau khuyến mãi, xếp theo đánh giá, top-3	không vượt ngân sách
InventoryAgent	id, kho → CHECK_STOCK/STOCK {số lượng, tuổi dữ liệu}	dữ liệu mới ($\leq$ 60 s)
Coordinator	mọi phản hồi → gọi theo thứ tự, kiểm hợp đồng, thử lại	đúng, không khẳng định sai, trễ

Mọi lời gọi đi qua `Coordinator.call` (Đoạn mã 8.1). Phản hồi chỉ được nhận khi đúng lớp `Message`, đúng người gửi/nhận, đúng loại, đủ trường; riêng STOCK còn phải có tuổi dữ liệu $\leq$ 60 s. Lỗi được ghi rồi thử lại; hết lượt thì trả `chua_xac_minh`, *không* khẳng định còn hàng. Độ trễ theo đồng hồ ảo: mỗi chặng trễ lognormal trung vị 2 ms cộng thời gian tính thật, quá hạn tốn trọn 80 ms.



Hình 8.1. Luồng thông điệp của hệ mua sắm: Coordinator gửi bốn yêu cầu (nét liền) và nhận bốn phản hồi (nét đứt) theo thứ tự 1–4; lời gọi kho lỗi được thử lại tối đa hai lần.

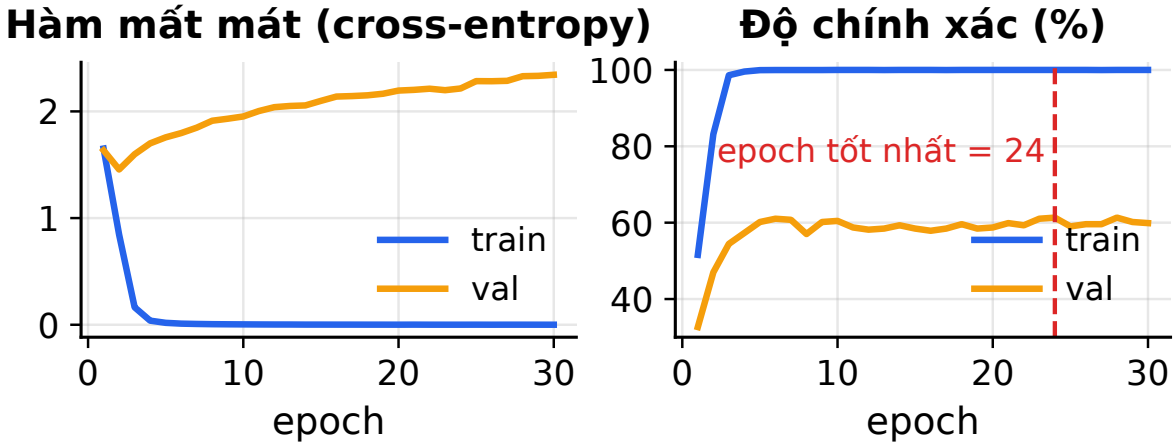
Đoạn mã 8.1. Coordinator gửi yêu cầu, kiểm hợp đồng phản hồi và thử lại khi lỗi.

```python
# ag2/shop_mas.py
def call(self, receiver, kind, payload, expect):
    """Gửi một yêu cầu, chờ phản hồi đúng hợp đồng; lỗi thì ghi sự kiện và thử lại. Hết lượt -> None."""
    for attempt in range(self.max_retries + 1):
        msg = Message("coordinator", receiver, kind, payload)
        self._hop(msg)
        t0 = time.perf_counter()
        try:
            resp = self.agents[receiver].handle(msg)
            self.clock += 1000 * (time.perf_counter() - t0)
            self.check(resp, receiver, expect)
            self._hop(resp)
            return resp.payload
        except (TimeoutError, ContractError) as e:
            self._fault(msg, e, attempt)
    return None
```

8.2 Bộ phân loại ý định bằng PyTorch trên GPU

Câu truy vấn được sinh tất định từ 14 khuôn câu Việt–Anh mỗi ý định, thêm biến thể như người dùng thật (hậu tố hội thoại, bỏ dấu 30 %, hoán đổi ký tự). Mấu chốt là **chia khuôn câu trước khi sinh câu** (9 train, 2 val, 3 test), nên test gồm những *cách nói chưa từng gặp*: 3.750 câu train, 349 val, 523 test, 6 ý định.

Mỗi câu là một túi đặc trưng (từ, cặp từ, 3-gram ký tự, ở cả dạng có dấu và không dấu) băm CRC32 vào 2^{16} ô theo tinh thần fastText [13]; `IntentNet` (Đoạn mã 8.2) lấy trung bình embedding rồi qua hai lớp tuyến tính, tổng 6.298.054 tham số. Huấn luyện AdamW [14] trên **NVIDIA GeForce RTX 4060 Laptop GPU**, 30 epoch trong 16,2 s, giữ trọng số epoch 24 có val accuracy cao nhất.



Hình 8.2. Đường học của IntentNet: train gần 100 % sau vài epoch, val dừng thấp hơn rõ rệt – khoảng cách đo đúng độ khó của khuôn câu mới.

Đoạn mã 8.2. Mạng phân loại ý định: EmbeddingBag trên đặc trưng băm và đầu phân loại hai lớp.

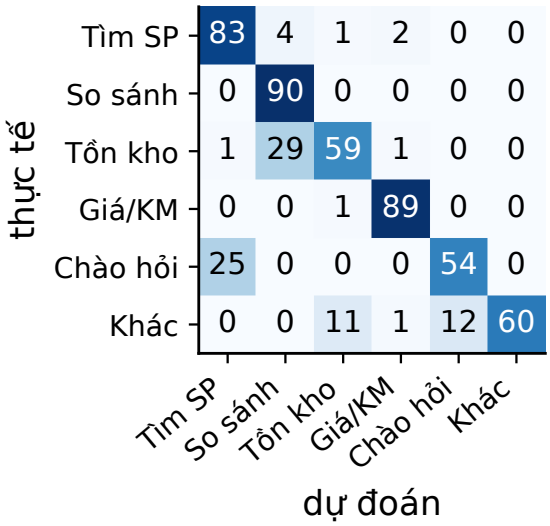
```python
# ag2/shop_intent.py
class IntentNet(nn.Module):
    """EmbeddingBag(trung bình) -> Dropout -> Linear -> ReLU -> Linear(6 ý định)."""

    def __init__(self, dim=96, hidden=64, n_classes=len(INTENTS), dropout=0.3):
        super().__init__()
        self.emb = nn.EmbeddingBag(N_BUCKETS, dim, mode="mean")
        self.head = nn.Sequential(nn.Dropout(dropout), nn.Linear(dim, hidden), nn.ReLU(), nn.Linear(hidden, n_classes))
```

```
9
10      def forward(self, ids, offsets):
11          return self.head(self.emb(ids, offsets))
```

Ý định	P	R	F1
Tìm SP	0,761	0,922	0,834
So sánh	0,732	1,000	0,845
Tồn kho	0,819	0,656	0,728
Giá/KM	0,957	0,989	0,973
Chào hỏi	0,818	0,684	0,745
Khác	1,000	0,714	0,833
Acc / Macro-F1		0,832	0,826

(a) Precision (P), recall (R), F1



(b) Ma trận nhầm lẫn

Hình 8.3. Bộ phân loại ý định trên tập test gồm các khuôn câu chưa gặp khi huấn luyện.

Nhận xét: Accuracy test 83,17 % (macro-F1 0,8264), thấp hơn hẳn train vì test đổi cách nói. Yếu nhất là *hỏi tồn kho* (recall 65,56 %, hay nhầm sang *so sánh*) và *chào hỏi* (recall 68,35 %), Hình 8.3b. Vì vậy cần ngưỡng tin cậy, chọn trên val: $\tau = 0{,}9$. Hệ nhận 74,57 % câu test, đúng 91,54 % trong số đó; còn lại được hỏi lại thay vì đoán sai.

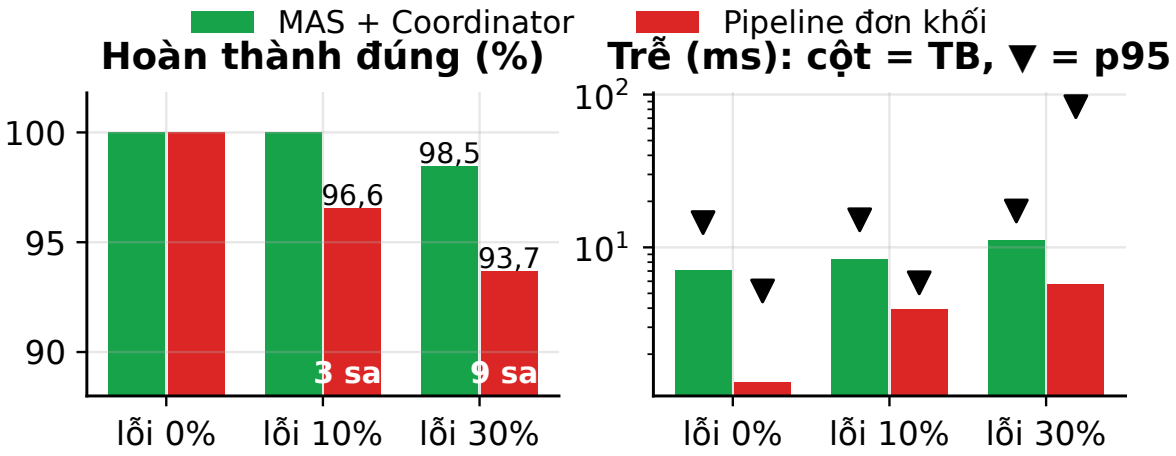
8.3 Demo, kiểm thử và mô phỏng lỗi

Hộp dưới là nhật ký thông điệp thật của một yêu cầu demo; lần gọi kho đầu được cài sẵn lỗi quá hạn nên Coordinator chờ hết 80 ms, ghi sự kiện, gửi lại và nhận tồn kho hợp lệ. 8 kiểm thử đơn vị (phân tích ngân sách, câu lạ, ngân sách, hàng hết, ba kiểu lỗi kho, sai người gửi, pipeline tin dữ liệu cũ) đều đạt.

Nhật ký thông điệp của Coordinator (`scripts/ch8_run.py`)

```
>>> Tìm giúp mình laptop dưới 20 triệu
[   2.5 ms] coordinator -> intent      CLASSIFY     {'query': 'Tìm giúp mình laptop dư
[  11.1 ms]      intent -> coordinator INTENT       {'intent': 'tim_san_pham', 'conf':
[  14.2 ms] coordinator -> catalog     SEARCH       {'query': 'Tìm giúp mình laptop dư
[  17.1 ms]     catalog -> coordinator CANDIDATES   {'category': 'laptop', 'brands': [
[  18.9 ms] coordinator -> recommend   RANK         {'ids': ['p01', 'p02', 'p03', 'p04
[  22.0 ms]   recommend -> coordinator RANKED       {'ids': ['p04', 'p07', 'p05']}
[  23.5 ms] coordinator -> inventory   CHECK_STOCK  {'ids': ['p04', 'p07', 'p05']}
[ 103.5 ms]  !! inventory: inventory không trả lời trước hạn chót (lần 1)
[ 105.3 ms] coordinator -> inventory   CHECK_STOCK  {'ids': ['p04', 'p07', 'p05']}
[ 106.9 ms]   inventory -> coordinator STOCK        {'stock': {'p04': 8, 'p07': 1, 'p0
=> ok p04, p07, p05 (9 thông điệp, 106,9 ms)
```

Để cô lập phần điều phối, cả hai hệ nhận *ý định đúng* của 523 câu test và phản hồi được so với đáp án chuẩn. Với xác suất 0, 10 hoặc 30 %, kho gặp một trong ba lỗi đều nhau: quá hạn, thiếu trường `stock`, hoặc bản chụp cũ 3.600 s lệch kho thật ở khoảng một phần ba mặt hàng. Pipeline đơn khối cùng logic nhưng gọi hàm trực tiếp, không kiểm hợp đồng, không thử lại.



Hình 8.4. Mô phỏng lỗi kho trên 523 yêu cầu: MAS có Coordinator so với pipeline đơn khối; nhãn trắng là số lần khẳng định sai tồn kho, trục trễ theo thang log.

Nhận xét: Khi kho không lỗi hai hệ cùng đúng 100,00 %, nhưng MAS tốn 3,335 thông điệp, 7,051 ms mỗi yêu cầu so với 1,312 ms: giá của phân vai. Ở mức lỗi 30 %, pipeline còn 93,69 %, **9 lần khẳng định sai** tồn kho và 29 yêu cầu hỏng; MAS giữ 98,47 %, không khẳng định sai lần nào, 8 yêu cầu báo "chưa xác minh". Có bộ phân loại ở đầu chuỗi, tỉ lệ đúng toàn trình là 68,26 %: nút thắt là hiểu câu.

Chương 9

Thảo luận, hạn chế và kết luận

9.1 Những quan sát chính

1. **Mốc chính xác thay đổi cách đọc kết quả.** Thưởng đánh giá lấy mẫu dao động theo chuỗi nhu cầu; giá trị chính xác của chính sách thì không. Ở baseline, RL đạt $34{,}8$ trên 10.000 bước, đúng bằng tác tử luật trên cùng chuỗi nhu cầu, và giá trị chính xác của cả hai là 35: hai con số khác 35 chỉ do may rủi của mẫu.
2. **"RL tốt hơn luật" không phải là một mệnh đề chung.** Ở $\delta = 20\%$ luật đã tối ưu, nên RL tốt nhất cũng chỉ hòa; RL trung bình còn thua nhẹ vì có seed học sai (chỉ $9/10$ seed đạt tối ưu). Ở $\delta = 5\%$, RL thắng. Ở $\delta = 10\%$, RL lại thua rõ (KTC $[-1{,}676;\ -0{,}9959]$) vì nó giảm giá ở nhu cầu thấp, tham lợi trước mắt, mà không thấy rằng chính hành động đó kích hoạt sự trả đũa của đối thủ khi nhu cầu cao.
3. **Trạng thái quan trọng hơn thuật toán.** Thêm $a_i(t-1)$ vào trạng thái giảm regret ở ba trên năm kịch bản của câu 10; học mô hình đối thủ rồi lập kế hoạch thì xử lý trọn lỗi trên: tác tử dựa trên mô hình có regret bằng 0 ở cả năm kịch bản.
4. **Siêu tham số quyết định độ ổn định.** $\alpha = 0{,}50$ làm chính sách tham lam đổi trung bình 1.752 lần trong huấn luyện; không khám phá ($\varepsilon = 0$) chỉ đạt tối ưu ở $3/10$ seed.
5. **Phần thưởng phải khớp mục tiêu kinh doanh.** Huấn luyện theo doanh thu làm lợi nhuận thật tụt còn $28{,}22$ mỗi bước.

9.2 Hạn chế

Mô hình cố ý đơn giản: hai mức nhu cầu, hai hành động, không tồn kho, không mùa vụ, khách hàng không nhớ giá. Kết quả chỉ có giá trị trong phạm vi mô hình và không nên suy rộng thành kết luận kinh tế. Mười seed là đủ để thấy độ phân

tán nhưng khoảng tin cậy vẫn rộng. Đề gốc và mã tham chiếu không có sẵn trên máy làm bài, nên mô hình được dựng lại từ công thức thống nhất của đề; mọi thay đổi về cách đo được ghi ở Chương 5. Hệ mua sắm ở Chương 8 dùng dữ liệu truy vấn tổng hợp và lời gọi đồng bộ trong một tiến trình.

9.3 Kết luận

Báo cáo đã mô hình hóa, cài đặt và đánh giá hai cách ra quyết định trong một hệ đa tác tử thương mại điện tử. Ba điểm rút ra: đánh giá phải tách khỏi huấn luyện và nên có mốc tối ưu chính xác; ưu thế của học hay của luật phụ thuộc vào thiết kế phần thưởng và vào việc trạng thái có đủ thông tin Markov hay không; và một tác tử dựa trên mô hình nhỏ, biết đối thủ phản ứng ra sao, vượt cả Q-learning lẫn luật cố định. Hướng tiếp theo là thêm tồn kho và nhiều mức giá, đối thủ có trí nhớ dài hơn, và đánh giá các thuật toán đa tác tử như học đồng thời có mô hình đối thủ.

Tài liệu tham khảo

[1] Học phần Phát triển các Hệ thống Thông minh. *Laboratory Experiment 02: Reinforcement-Learning-Based Agents versus Rule-Based Agents*. Tệp assignment_Agent02_due_29.10.pdf, năm học 2026–2027. Học viện Công nghệ Bưu chính Viễn thông. 2026.

[2] Học phần Phát triển các hệ thống thông minh. *Developing Multiagent Systems: A Practical Tutorial with Python and PyTorch*. Tài liệu nội bộ học phần, mục 1–19. Học viện Công nghệ Bưu chính Viễn thông, Khoa CNTT 1. 2026.

[3] Muddasar Naeem, Syed Tahir Hussain Rizvi, and Antonio Coronato. "A Gentle Introduction to Reinforcement Learning and its Application in Different Fields". In: *IEEE Access* 8 (2020), pp. 209320–209344. DOI: 10.1109/ACCESS.2020.3038605.

[4] Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O'Sullivan, and Hoang D. Nguyen. *Multi-Agent Collaboration Mechanisms: A Survey of LLMs*. 2025. arXiv: 2501.06322 [cs.AI]. URL: https://arxiv.org/abs/2501.06322.

[5] Michael Wooldridge. *An Introduction to MultiAgent Systems*. 2nd ed. John Wiley & Sons, 2009.

[6] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2021.

[7] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018. URL: http://incompleteideas.net/book/the-book-2nd.html.

[8] Christopher J. C. H. Watkins and Peter Dayan. "Q-learning". In: *Machine Learning* 8.3–4 (1992), pp. 279–292.

[9] Volodymyr Mnih et al. "Human-level control through deep reinforcement learning". In: *Nature* 518 (2015), pp. 529–533. URL: https://www.nature.com/articles/nature14236.

[10] Lucian Buşoniu, Robert Babuška, and Bart De Schutter. "A Comprehensive Survey of Multiagent Reinforcement Learning". In: *IEEE Transactions on Systems, Man, and Cybernetics, Part C* 38.2 (2008), pp. 156–172. DOI: 10.1109/TSMCC.2007.913919.

[11] Học phần Phát triển các Hệ thống Thông minh. *AGENT02: Python Implementation – Reinforcement-Learning-Based Agent vs Rule-Based Agent*. Mã tham chiếu agent02_experiment.py. Học viện Công nghệ Bưu chính Viễn thông. 2026.

[12] Học phần Phát triển các Hệ thống Thông minh. *Developing Multiagent Systems: A Practical Tutorial with Python and PyTorch*. Tài liệu nội bộ học phần, tệp slide_multiAgent_system_20.10.2026.pdf. 2026.

[13] Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. "Bag of Tricks for Efficient Text Classification". In: *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*. 2017, pp. 427–431.

[14] Adam Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems 32 (NeurIPS)*. 2019, pp. 8024–8035.